

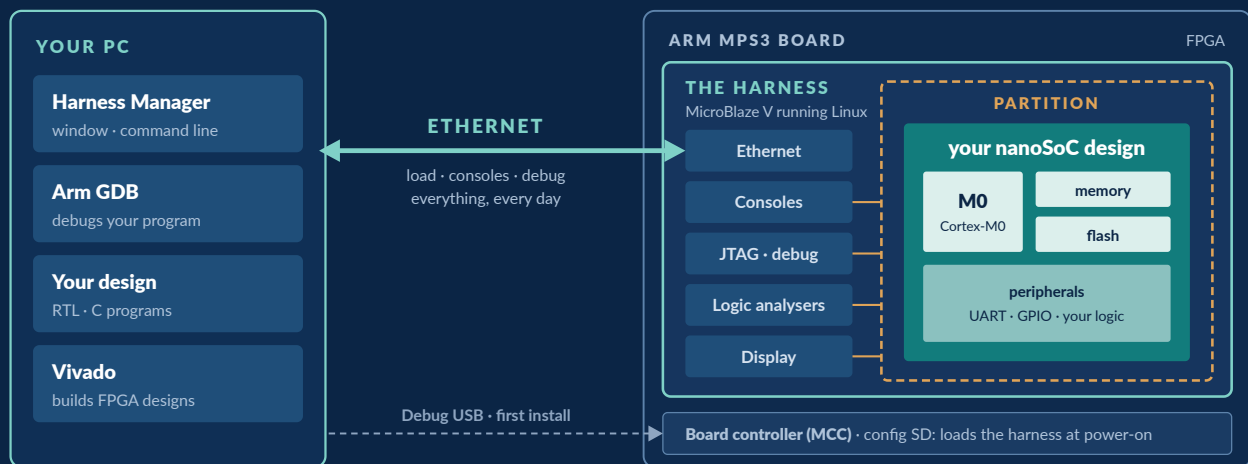
TECHNICAL REFERENCE & MAINTAINERS GUIDE

nanoSoC:

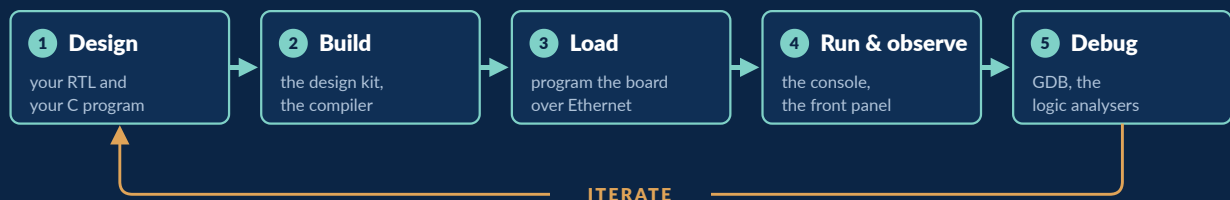
MPS3 Verification Platform

How to commission, run, update and recover MPS3 boards on your own site, and how the harness inside them is built, so that you can change it safely.

THE PLATFORM



HOW YOU WORK ON IT



Platform v2.0 · Harness Manager 1.0.0 · 7 October 2026

Companions: *Getting Started* and the *User Guide*.

SoC Labs. Arm IP (Cortex-M processors, CoreSight, CMSDK) is used under licence from Arm and is not part of the platform.

Introduction

This guide is for the people who look after MPS3 boards running the platform, and for the people who change the harness itself. It assumes you have used the platform as *Getting Started* describes, and it points to the *User Guide* for everyday tasks rather than repeating them.

Part 1, operating and maintaining boards, is for a site or infrastructure team running its own boards: one on a desk, or several in a lab. By the end of it you can commission a new board, keep it on the current release, run several boards on one network, check that they stay healthy, and bring a board back when it stops answering. Running a shared hub for a group comes last, in chapter 6, because a board of your own needs none.

Part 2, how the harness works, is for people who modify the harness or need to understand the FPGA design underneath: the hardware, the static shell, the harness Linux, the partition, the on-board debug path, the build and release flow, and the rules for changing any of it without breaking the designs that depend on it.

What you need first

- An Arm MPS3 board (V2M-MPS3, revision C), its power supply, its Debug USB cable and an Ethernet cable.
- A PC running Linux, macOS or Windows with Harness Manager 1.0.0, installed as *Getting Started* ch 6 describes.
- The platform release from <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>. It carries the open designs only: users with Arm Academic Access build the designs that contain Arm IP themselves (*User Guide* §7.7).
- For Part 2, and for a few Part 1 tools: a clone of the platform's source repository, which is public on GitHub at <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform> (check out the release tag, `v2.0.0`), and AMD Vivado 2026.1 for anything that builds FPGA logic.

Conventions

YOU SEE	IT MEANS
<code>B=192.168.10.101</code> , <code>\$B</code>	The board's address. Every new board answers on <code>192.168.10.101</code> ; chapter 3 gives each board its own.
<code>harness-manager ...</code>	Harness Manager's command line. Every command and flag here is checked against its <code>--help</code> .
<code>pyverify ...</code>	The platform's protocol tool, which Harness Manager installs in its own environment. Define it once: <code>pyverify() { ~/.local/share/harness-manager/venv/bin/python -m pyverify.cli "\$@"; }</code> (Linux and macOS).
<code>fpga/dfx/...</code> , <code>make -C fpga/dfx ...</code>	A path, or a command run, at the root of your clone of the platform repository (https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform). <code><checkout></code> stands for that directory.
<code><board></code> , <code><zip></code>	Something you fill in.
<code>§9.2</code> ; <i>User Guide</i> ch 2	A section of this guide; a chapter of another document.

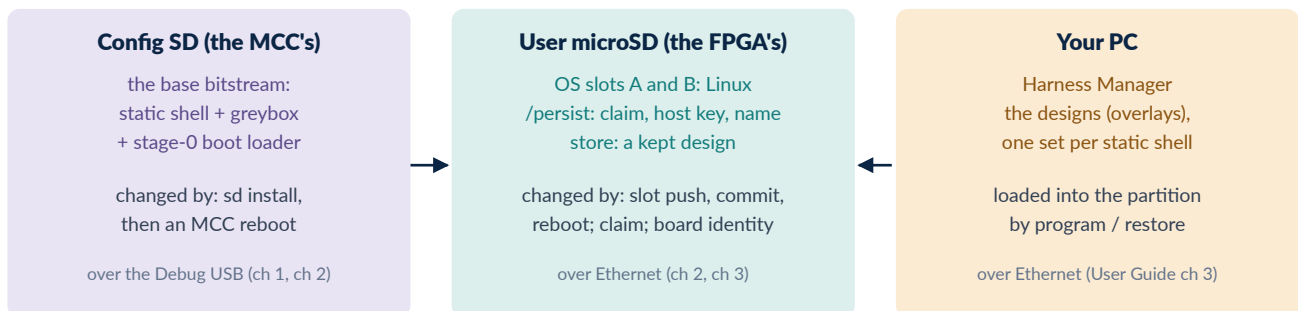
For help, contact SoC Labs support (the support contact that came with your board).

The platform at a glance, and why

The platform turns an MPS3 into a board you can give a new system-on-chip over Ethernet, without touching it, and then debug from a desk anywhere. It does this by splitting the FPGA in two: a permanent **harness**, and a **partition** that holds one design at a time.

The pieces, and why each exists

- **The MPS3's own controller (the MCC)** can only load a whole FPGA bitstream from its config SD card, which takes minutes and stops everything. So it is used once, to load the harness, and after that only for recovery.
- **The static shell** is the permanent half of the FPGA: a soft processor, the Ethernet controller, the engine that rewrites the partition, the debug bridges and the front-panel display. It is built once per release and locked, so that every design meets it on exactly the same wires.
- **The harness Linux** runs on the shell's processor (a MicroBlaze V). Its services answer the network, load designs, serve consoles and debug, and drive the panel. Linux lives on the user microSD, so it can be updated without touching the FPGA.
- **The partition** holds the design under test, usually a nanoSoC: SoC Labs' family of Arm Cortex-M SoCs built on an AHB bus fabric (*Getting Started* ch 3). It is rewritten over Ethernet in about a minute while the harness keeps running, so a design that hangs, or was built wrong, can always be swapped out.
- **Harness Manager**, on your PC, is the window and command line that drive all of this. It needs the board's Debug USB once, to install the harness; after that, only Ethernet.



Power-on: the MCC loads the base from the config SD; stage-0 boots Linux from an OS slot; Linux loads the design kept in the store, if any.

Figure G.1. Where a board's software lives. Each of the three places is changed a different way, which is why the maintenance chapters are organised around them.

The numbers

Board	Arm MPS3 (V2M-MPS3), revision C; revision B boots but is untested. AMD Kintex UltraScale XCKU115 FPGA.
Harness	Platform v2.0: static shell 0x44EE76D5, MicroBlaze V running Linux, built with Vivado 2026.1.
Partition	42 824 LUTs and 144 block-RAM tiles; one design at a time; 47 boundary ports, 148 bits.
Times	Power-on to SSH 3 to 4 minutes; a design load about a minute (35–86 s); a Linux image into an OS slot 41 to 50 minutes.
Link	One 10/100 Ethernet port carries control, loads, consoles, debug, logic analysers and SSH.

Eight principles

Most design decisions in Part 2 follow from these. Each is enforced by the build, a check or the harness itself, because breaking it once cost the project something. When you change the harness, keep them.

PRINCIPLE	WHY	ENFORCED BY
1 Reload only the half that changes	The MCC's whole-FPGA load is paid once. Afterwards only the partition is rewritten, and the harness keeps its network, sessions and state.	The DFX build (§12.3); the swap (§10.3)
2 Everything that reaches the board is static	Network, processor, loader, debug and consoles are all in the shell, so no design can cut you off. The partition is never empty: it holds <code>greybox</code> , an inert tie-off.	The shell block design; <code>greybox</code> first in the build's design list
3 A narrow, frozen boundary	Shell and design meet at 47 scalar ports and byte streams: no bus, no clock generation, no pads. A wrapper that differs by one bit fails late, so the boundary is checked in a second before any tool runs.	<code>pin_check.py</code> ; Vivado <code>pr_verify</code>
4 Ethernet is the only wire	After the first install, everything shares one 10/100 port, so a board can sit in a rack and be used from anywhere. The price is bandwidth.	The port map (<i>User Guide</i> ref C)
5 Identity is read back, never assumed	A design names the shell it was built for and is refused elsewhere; after a load the harness reads the design's id back; the panel shows the live one.	The <code>static_id</code> guard in every tool; the identity lock (§10.5)
6 Frozen contracts between parts	The boundary, the register map, the network protocol and the overlay manifest are written down and versioned, so each side can change without guessing about the other.	<code>docs/contracts/</code> ; generated views with freshness checks (§12.7)
7 Evidence over assertion	A check that cannot fail is not a check; a stage that did not run says so; a claim that something works on the board cites a log.	The repository gate (§12.7)
8 One SoC description everywhere; vendor IP outside	A nanoSoC is described once and generated for simulation, the FPGA and silicon. Arm IP is read in place, under its own licence, and never copied into the platform.	The SoC generator (§12.2)

What it is not. It is not a speed-accurate prototype: the design runs at tens of MHz and proves function, not performance. It is not a replacement for simulation. And it is not a general-purpose FPGA board: a design that needs a bus into the shell or its own clocking needs a boundary change, which means a new static shell (§13.2).

Contents

PART 1 • OPERATING AND MAINTAINING BOARDS 7

1	Commissioning a board	7
1.1	What you need	7
1.2	Check the revision and the controller	7
1.3	Install the harness on the config SD	8
1.4	Put Linux on the user microSD	8
1.5	Claim the board	10
1.6	Give it its name, MAC and address	10
1.7	Acceptance checks	10
2	Releases and updates	12
2.1	Where releases come from	12
2.2	What a release contains, and how each part reaches a board	12
2.3	Reading a release's identity	13
2.4	The OS slots: installing a new Linux image	13
2.5	Updating to v2.0.1 or later	14
2.6	A release that changes the static shell	14
2.7	Updating Harness Manager	14
2.8	Going back	14
3	Several boards on a network	15
3.1	Every new board looks the same	15
3.2	How a board joins a network	15
3.3	Why each board needs its own MAC and address	15
3.4	Giving each board its identity	16
3.5	Laying out a site	16
3.6	Finding a board, and telling boards apart	17
3.7	Several people, one board	17
4	Health checks and soak tests	18
4.1	The quick check	18
4.2	Harness Manager's overnight checks	18
4.3	The soak	19
5	The recovery ladder	21
5.1	Ask the board first	21
5.2	The ladder	21
5.3	Rescue, and Linux from your PC	22
5.4	A dark board: read the FPGA over JTAG	22
5.5	Loading the harness over JTAG	23
5.6	The MCC: pacing, and what never to send	23
5.7	The harness serial console, and releasing a claim	24
6	Running a hub for a group	25
6.1	What a hub is	25

6.2	Setting up the hub	25
6.3	The claim on a shared board	26
6.4	Leases	26
6.5	Config-SD writes and MCC reboots through the hub	26
6.6	Checks, soaks and boot rates on a hub	27
6.7	Hub traps	27

PART 2 • HOW THE HARNESS WORKS 28

7	Hardware and technologies	28
7.1	Dynamic Function eXchange	28
7.2	The configuration port, the processor and the debug links	28
7.3	The board and its FPGA	29
7.4	How the board boots	29
7.5	Clocks	30
7.6	Who owns each interface	30
8	The static shell	32
8.1	The blocks	32
8.2	The registers that matter most	32
8.3	The design's Ethernet bench	33
8.4	The front panel	33
9	The harness Linux	34
9.1	The layers	34
9.2	How it boots	34
9.3	The user microSD	35
9.4	One service, thirteen tasks	35
9.5	SSH, the claim and the claim lock	36
9.6	The board's identity	36
9.7	Ports and protocols	36
9.8	Mailboxes and logs	37
10	The partition	38
10.1	The boundary and the partition	38
10.2	The overlay	38
10.3	A load, step by step	39
10.4	Fencing the partition	39
10.5	Identity	39
10.6	Timing at the boundary	40
10.7	Keeping a design on the card	40
11	The on-board debug path	42
11.1	The chain	42
11.2	OpenOCD on the board: mps3-debug	42
11.3	How Harness Manager drives it	42
11.4	Speed, and what comes next	43

Operate	Maintain	Recover	Architecture	Build	Change
11.5	Logic analysers over XVC	43	13.3	Changing the register map	49
11.6	An OpenOCD on the PC	43	13.4	Changing the protocol	49
11.7	Rules	43	13.5	Traps when building and changing	49
12	Building and releasing the harness	44	14	Harness Manager architecture	51
12.1	The environment	44	14.1	Design rules	51
12.2	The SoC generator	44	A	Pin reference	52
12.3	The build targets	44	B	The address map	54
12.4	Building a new static shell	45	C	The bare-metal shell (v1.x)	55
12.5	The Linux image and the release bundle	46	D	Platform-developer troubleshooting	56
12.6	The build kit	46	D.1	The harness	56
12.7	Verification before a release	47	D.2	Scripts and sessions	56
13	Changing the harness safely	48	D.3	The nanoSoC designs and the design's flash	57
13.1	What each kind of change costs	48			
13.2	Changing the boundary	48			

PART 1 · OPERATING AND MAINTAINING BOARDS

1 Commissioning a board

This chapter takes a new MPS3 from its box to a board in service: the right revision and controller, the harness on its config SD, Linux on its user microSD, a claim, its own name and address, and a set of checks you record. Allow about an hour and a half with a card reader, or two and a half hours without one, most of it waiting.

#	STEP	NEEDS	TIME
1	Check the revision and the controller (§1.2)	The config SD on your PC	5 min
2	Install the harness on the config SD (§1.3)	Debug USB	15 min
3	Put Linux on the user microSD (§1.4)	A card reader, or Ethernet	15 min, or 75 min
4	Claim the board (§1.5)	Ethernet	10 min
5	Give it its name, MAC and address (§1.6)	Ethernet	5 min
6	Run the acceptance checks and record them (§1.7)	Ethernet	10 min

1.1 What you need

- **The board** with its power supply, its Debug USB cable (for steps 1 and 2 and for recovery) and an Ethernet cable.
- **A microSD card of 4 to 32 GB** (SDHC) for the user microSD slot. Cards of 2 GB or less are refused; larger cards are untested. A board in service needs one: without it, Linux must be pushed again after every reset and the board keeps no claim, name or design.
- **A PC** with Harness Manager 1.0.0 installed with `--with-serial` (*Getting Started* ch 6). On Linux your user must be in the serial ports' group (`dialout` on most distributions); otherwise `mcc` stops with `no permission to open serial port ... add your user to the port's group`.
- **The release** from <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>: the config-SD bundle, the Linux image with its `linux_bundle.json`, the card tool `stage0_mkcard.py`, and the open designs.
- **A record sheet** for the board: serial number, revision, MCC version, the backup's file name and sha256, label, MAC, IP, the SSH host-key fingerprint, and the results of §1.7.

1.2 Check the revision and the controller

The release supports **revision C** boards (motherboard HBI0309C). Revision B boards boot the same release, because the config-SD bundle carries a matching folder for them, but they are untested. There is no support for revision A. Check before you install, so that you know which you are commissioning.

- 1 **Read the silkscreen.** Near the FPGA's fan the board reads `HBI-0309C` or `HBI-0309B`.
- 2 **Read the controller's log.** Power the board with the Debug USB connected; the config SD appears on your PC as a drive named `V2M-MPS3`. Its `LOG.TXT` records the board, for example `MotherBoard Revision C Variant A`, and the MCC firmware version. "Variant A" is not revision A.
- 3 **Record the MCC version, and leave it alone.** MCC firmware v1.3.2 is the version the platform is proven on; other versions are allowed but untested. Do not update or downgrade the MCC to match: Harness Manager never writes the MCC's firmware, and the platform does not need it changed.

NOTE

Arm documents three differences between revisions B and C: the debug reset on the DAPLink connector (nTRST on B, nSRST on C), the DAPLink virtual serial port to an FPGA UART (C only), and changes to the Arduino shield interfaces. The harness uses none of them, which is why a revision B board is expected to work; it has simply not been tested (Arm, *MPS3 Getting Started Guide*, 107789).

1.3 Install the harness on the config SD

The config SD holds the base bitstream the MCC loads at power-on: the static shell with its boot loader, stage-0, inside. Installing it is the only step that needs the Debug USB. *Getting Started* ch 7 walks a first-time owner through it; the commands are these.

```
harness-manager probe # the board, its MCC port and the V2M-MPS3 volume
harness-manager sd - --volume /media/$USER/V2M-MPS3 backup ~/mps3-backups
harness-manager sd - --volume /media/$USER/V2M-MPS3 install ~/mps3-bundle/config-sd \
--backup ~/mps3-backups/<zip>
harness-manager mcc - --serial /dev/ttyUSB0 reboot
```

```
backed up 10 files from V2M-MPS3 to .../V2M-MPS3-20261002T125025Z.zip
sha256 a2fa5a92...
...
written 4 files to the SD of mps3@usb:/media/.../V2M-MPS3
not running yet: reboot (`harness-manager mcc TARGET reboot`), then check `harness-manager info TARGET`
...
rebooted mps3@usb:serial:///dev/ttyUSB0 (seen: sent, down, up)
MCC loaded MB/HBI0309C/Nanosoc/nanosoc.bit
```

- **The backup is your way back** to the board as it came (§5.2, step 7). `install` refuses without one. Copy the zip and its `.sha256` off the PC into your site's records.
- **The install takes up to 5 minutes.** Slow is not failed: never unplug the cable, power off or start a second write. An interrupted install leaves a marker, and the next install refuses until you restore the backup.
- **The reboot is paced.** The MCC drops characters sent in a burst, so Harness Manager sends its commands slowly and then watches the board go down and come back (§5.6).

Then connect the board's Ethernet port to your PC, give the PC an address in `192.168.10.0/24` (such as `192.168.10.1`), and ask the board:

```
B=192.168.10.101
harness-manager info $B
```

```
board      MPS3 at 192.168.10.101:6900
shell      0x44ee76d5  harness ? (?)
...
control    rescue
note       stage0 is in RESCUE: ... The microSD holds no bootable OS slot ...
```

`rescue` is right at this point: the harness is in the FPGA and waits for Linux. Record the `shell` value.

1.4 Put Linux on the user microSD

Stage-0 boots Linux from one of two OS slots on the user microSD (§9.3). There are two routes to a first card. Use the first if you have a card reader; it is quicker.

WITH A CARD READER

This route uses the same layout tool as the network route below, but has not yet been run end to end on a board in this release. If slot A does not boot, use the route without a reader.

- 1 **Make the card image** in the release directory, and check it the way the boot loader will read it:

```
python3 stage0_mkcard.py card --slot-a linux_slot.img --out-dir card \
  --card-img card.img --card-mib 512
python3 stage0_mkcard.py check card.img      # ends: -> stage0 would boot slot A
```

- 2 **Write `card.img` to the microSD** with `dd` or a raw image writer (*Getting Started* ch 8 has the commands for each system). It erases the card. Check the device name twice: never your PC's own disk, and never the config SD.
- 3 **Boot from it.** Switch off, insert the card in the user microSD slot (not the config SD's), switch on, and wait 3 to 4 minutes. `harness-manager info $B` now shows the control state `idle`, and `harness-manager slot status $B` shows slot A running.

WITHOUT A CARD READER

The board can lay out its own card over Ethernet. This route was run end to end on a real board for this release. It takes about 75 minutes, most of it the slot push. *User Guide* ch 2 explains each step; in outline:

- 1 **Start Linux from your PC.** In rescue, stage-0 takes a Linux image over the network and boots it once, from memory. The push tool is `stage0_push.py` in the platform repository (`src/linux_soc/hw/fw_stage0/`, with `stage0_pack.py` beside it); clone <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform> at the release tag to get it:

```
pyverify identify --host $B                                # "mode": "rescue"
pyverify netboot ~/mps3-bundle/linux_slot.img --host $B \
  --tool <checkout>/src/linux_soc/hw/fw_stage0/stage0_push.py # exit 0: accepted
pyverify identify --host $B                                # about 20 s later: "mode":
"run"
```

- 2 **Claim it for now** (§1.5). Linux running from memory keeps the claim only until the reboot in step 5.
- 3 **Lay out the card and format `/persist`**, one command after another with no restart in between: on a reused card, a restart before the format would bring back the old card's claim.

```
python3 stage0_mkcard.py card --slot-a ~/mps3-bundle/linux_slot.img --out-dir ~/mps3-card
harness-manager board ssh $B -c 'dd of=/dev/mmcblk0 bs=512 count=1 conv=fsync' < ~/mps3-card/mbr.bin
harness-manager board ssh $B -c 'dd of=/dev/mmcblk0 bs=512 seek=1 count=2 conv=fsync' <
~/mps3-card/bootsel.bin
harness-manager board ssh $B -c 'partprobe /dev/mmcblk0 && mps3-persist format --erase'
```

- 4 **Push Linux into a slot, and commit it.** The board writes the image and reads it back: 41 to 50 minutes. It lands in slot B, because slot A is the default while the board runs from memory.

```
harness-manager slot push $B --bundle ~/mps3-bundle
harness-manager slot commit $B
```

5

Reboot onto the card (`harness-manager mcc - --serial /dev/ttyUSB0 reboot`), then `slot status`: slot B running and the default.

1.5 Claim the board

A claim decides which SSH keys the board trusts. Until it is claimed, a Linux harness takes no key at all; the first key it is sent claims it for good, and only a person at the serial console can release it (§5.7). On a claimed board, SSH, JTAG, the logic-analyser port and every change to the user microSD or the board's identity go through those keys, and Harness Manager uses SSH for you.

- **Claim with the key you will keep.** For a board the site team runs for several people, claim with a keys file (one public key per line) holding the team's keys and those of the people who will debug on it. Harness Manager's confirmation shows only the first key of a multi-key file; all of them are sent.
- **A new card takes two claims.** `/persist`, where the claim and the SSH host key live, is not formatted on a new card, so the first claim lasts only until the next restart. Format `/persist` (the no-reader route already did), restart, and claim again; the board then keeps a new host key, which you accept once.

```
harness-manager board claim $B --key ~/.ssh/id_ed25519.pub
harness-manager board ssh $B -c 'mps3-persist status'      # backing=card ... : done; backing=tmpfs:
format, restart, claim again
harness-manager board ssh $B -c 'mps3-persist format --erase'
pyverify reboot --host $B --wait
harness-manager board claim $B --key ~/.ssh/id_ed25519.pub --replace-host-key
```

Claim only while `info` shows the harness running: in rescue, stage-0 takes any upload as a boot image. If the board was claimed from another PC with your key, pin that claim with `harness-manager board claim $B --adopt`. Record the host-key fingerprint that `harness-manager board claim-status $B` shows.

1.6 Give it its name, MAC and address

Every new board answers as `MPS3` at `192.168.10.101` with the MAC `02:00:00:4D:50:53`. That is fine for one board on a desk, and wrong for two on one network (chapter 3 explains why). Give each board its own identity now, while it is on your bench:

```
harness-manager board identity $B --label MPS3-LAB4 --mac 02:5a:91:3c:07:e2 --ip 192.168.10.104/24
```

- **The label** is the name on the front panel's top row and, in lower case, the host name: up to 16 characters of `A–Z`, `0–9` and `-`, so that it fits beside the panel's status badge.
- **The MAC** must be unique on your network. Use a locally administered address (first byte `02`) and generate the rest at random (§3.4).
- **The IP** is the static address the board always answers on, beside any DHCP lease (§3.2).

It needs the claim and `/persist` on the card. Harness Manager asks you to type the new label, writes the setting, restarts the harness (about 3 minutes) and reads the identity back. If you changed the address, the board stops answering on the old one: reach it at the new one, and move your PC's address too if you changed the subnet.

1.7 Acceptance checks

Run these before you hand the board over, and record each result on its sheet. They prove the network, the load path, the consoles, the card and the panel.

RUN	HEALTHY RESULT
<code>harness-manager info \$B</code>	The new label, <code>shell 0x44ee76d5</code> , control <code>idle</code> , <code>build firmware/fabric check: ok</code>
<code>harness-manager slot status \$B</code>	One slot running and the default, confirmed
<code>harness-manager card status \$B</code>	A card present; its store formatted or empty
<code>harness-manager identify \$B</code>	The panel's backlight blinks with an IDENTIFY banner: you have the right board
<code>harness-manager program \$B led</code>	<code>verified</code> , in about 35 s; the board's LEDs count up
<code>harness-manager program \$B uart_echo</code> , then <code>harness-manager console \$B uart0</code>	The banner <code>rm_uart_echo ready</code> after a DUT reset (Ctrl-] then r, r); what you type comes back. Ctrl-] exits
<code>harness-manager restore \$B</code>	<code>restored ... to the baseline (0x00000000) ... verified</code> , in about 40 s

If you have built `nanosoc` (*Getting Started* ch 11), also load it with its console open first, run `harness-manager debug up $B`, attach Arm GDB with the line it prints and halt the core (*Getting Started* ch 12). Leave the board on `greybox` when you finish. For a board that will run unattended, add a soak (§4.3).

2 Releases and updates

A release changes up to four things, and each reaches a board a different way. This chapter says where releases come from, what is in one, how to tell which parts a new release changes, and how to install it, check it and go back.

2.1 Where releases come from

- **The platform release** (the harness bundle, the open designs and the FPGA build kit) is published on GitHub: <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>. The platform's source repository, for building or changing the harness (Part 2), is public too: <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform>. Each release has release notes, which list its known limitations and anything special about installing it.
- **Harness Manager** is published at <https://github.com/SoC-Labs/HarnessManager>.
- **The designs that contain Arm IP** (`nanosoc`, `nanosoc_upy`, `nanosoc_multicore`, `eth_ss`, `nanosoc_ila`) are **not in the release**. A user with Arm Academic Access builds them with Harness Manager's kit from the nanoSoC source, which comes from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support) (*User Guide* §7.7). The platform repository holds their wrappers, as it holds all of the platform's source.

Watch the releases page, and read the release notes before you update a board: they say which of the parts below changed. When a release changes the static shell, your users rebuild their nanoSoC designs with the new kit, as they rebuild their own.

2.2 What a release contains, and how each part reaches a board

PART	WHAT IT IS	HOW IT REACHES A BOARD
Config-SD bundle	<code>config-sd/</code> , laid out like the config SD: <code>config.txt</code> , and under <code>MB/</code> one folder for revision C boards (<code>HBI0309C</code>) and an identical one for revision B (<code>HBI0309B</code>). It holds the base bitstream: the static shell with stage-0 inside.	<code>sd install</code> over the Debug USB, then an MCC reboot (§1.3). The MCC picks the folder that matches the board.
Linux image	<code>linux_slot.img</code> , with <code>linux_bundle.json</code> naming the static shell it was built for and its sha256; <code>linux_slot_netboot.img</code> , a network-boot build of the same Linux (the procedures here use <code>linux_slot.img</code>); and <code>linux_legal_info.tar</code> , the open-source licence information for both.	Over Ethernet, into the free OS slot: push, commit, reboot (§2.4).
Designs (overlays)	<code>overlays/</code> : the eight open designs (<code>greybox</code> , <code>led</code> , <code>regdemo_a</code> , <code>regdemo_b</code> , <code>uart_echo</code> , <code>clcd_demo</code> , <code>dbg_demo</code> , <code>socscope</code>), one folder each with <code>manifest.json</code> (naming the static shell it fits), <code><name>.bin</code> (the partial) and <code><name>_clear.bin</code> (the clearing). No design that contains Arm IP: users build those (§2.1).	Copied to your PC; Harness Manager loads them over Ethernet (<i>User Guide</i> ch 3).
Build kit	One zip for the release's static shell: the locked static checkpoint, the boundary and the exact Vivado release (§12.6).	On the PC where your users build their own designs; Harness Manager imports it (<code>kit import</code>).

PART	WHAT IT IS	HOW IT REACHES A BOARD
Tools	The card tools <code>stage0_mkcard.py</code> and <code>stage0_pack.py</code> ; Harness Manager, released alongside.	On your PC (§2.7).
Checksums	<code>MANIFEST.txt</code> , the bundle's list of files, and <code>SHA256SUMS</code> .	Check the download before you install anything: <code>sha256sum -c SHA256SUMS</code> in the bundle's folder.

2.3 Reading a release's identity

Two numbers decide what an update involves:

- **The `static_id`** (v2.0: `0x44EE76D5`) names the locked static shell. A design fits only the shell it was built for. Every load is checked against it, and a design for another shell is refused before a byte reaches the FPGA.
- **The `release tag`** (v2.0.0) names the release as a whole.

If a new release keeps the `static_id`, every design you have, including your users' own builds, keeps working, and the update is usually a new Linux image only. If it changes the `static_id`, every design must be rebuilt for the new shell: the release then brings a new set of designs and a new build kit, attached to the release, and your users rebuild theirs with it. The release notes say which.

The board also reports a harness version stamped into the bitstream (shown by `info` as `harness 1.0.0` on v2.0). It changes only when the static shell is rebuilt, so it can differ from the release tag. Use the `static_id` and the tag.

2.4 The OS slots: installing a new Linux image

The user microSD holds two OS slots, A and B. A new image is written into the slot that is neither running nor the default, read back by the board, and boots only after you commit it and reboot. If it never boots healthily, stage-0 goes back to the other slot by itself. This makes a Linux update safe to do remotely.

- 1 Look first.** `harness-manager slot status $B` shows which slot runs, which is the default and where a push will go.
- 2 Push the image.** The board checks that the image was built for its static shell (exit 14 if not), writes it and reads it back: 41 to 50 minutes for the 29 MB image. Leave it to finish; the board keeps working meanwhile.

```
harness-manager slot push $B --bundle ~/mps3-bundle
```

```
push: writing slot B: 12.2 MB / 32.6 MB, ~47 min left (40 KB/s)
...
done      slot B holds linux_slot.img, read back; `harness-manager slot commit
192.168.10.101` makes it boot next
```

- 3 Commit it:** `harness-manager slot commit $B`. After a commit and before the reboot no slot is free; `slot push --rollback-first` undoes a commit you change your mind about.
- 4 Reboot.** In the window, Board → Recover → Reboot via the MCC; or, with the Debug USB, `harness-manager mcc - --serial <port> reboot`; or a warm restart, `pyverify reboot --host $B --wait`. Linux is back in 3 to 4 minutes.
- 5 Check it.** `slot status` shows the new slot running and the default. It says `booted` (not yet confirmed) until the harness confirms a healthy boot (network up, SSH listening, card readable). Then load `greybox` once (`harness-manager restore $B`): after any restart the partition stays in reset until the first load.

2.5 Updating to v2.0.1 or later

v2.0.1 keeps the v2.0 static shell and changes only the Linux image: its on-board debugger reads and writes the design's memory much faster than v2.0.0's (§11.4). Install it as §2.4 describes; no config-SD write, no new designs and no Harness Manager update are needed. Install v2.0.1 or later on any board people use for software debugging.

2.6 A release that changes the static shell

A new `static_id` means a new base on the config SD, a Linux image built for it, and a new set of designs. All three must move together: an image built for one shell refuses to load designs on another (it reports an *identity lock*), and designs built for the old shell are refused by the new one. The release notes of such a release give the order to install them in. Plan it as a maintenance window: tell your users, point them to the release's new designs and build kit, and keep the config-SD backup that `sd install` makes, which is your way back.

2.7 Updating Harness Manager

Update Harness Manager on each PC with the new release from <https://github.com/SoC-Labs/HarnessManager>: in your clone, `git pull` (or check out the release tag), then run the installer again with

the same options. It keeps your settings, your board list and your config-SD backups.

```
git -C HarnessManager pull
HarnessManager/scripts/install.sh --with-serial
harness-manager version
```

Harness Manager also has an in-app update channel (Board → Versions, and `harness-manager update`, `harness-manager harness`) for signed releases. Harness Manager 1.0.0 carries SoC Labs' release-signing keys, but only a `beta` channel is published: the default `stable` channel answers `not found (404)` (exit 7). Until a stable channel is published, update by hand as above, or add `--channel beta`.

2.8 Going back

TO UNDO	DO
A new Linux image	<code>harness-manager slot rollback \$B</code> : the other slot becomes the default and the board reboots into it (up to 180 s; <code>--no-reboot</code> waits for the next reboot). A slot written outside Harness Manager, such as by the card tool, has no slot record, so <code>slot rollback</code> and <code>slot verify</code> refuse it.
A config-SD install	<code>harness-manager sd - --volume <V2M-MPS3> restore <zip></code> , then an MCC reboot (§5.2, step 7). The zip is the backup that install made.
Harness Manager	Check out the previous release tag in your clone and run the installer again.

Keep, for every board, the release it runs, the date of each install, and the backup zip each install made. When a board misbehaves after an update, those three facts decide what to roll back.

3 Several boards on a network

One board on a desk can keep its factory identity. Two or more on one network cannot. This chapter explains how a board joins a network, why each board needs its own MAC and address, three ways to lay out a site, and how to tell the boards apart once they are running.

3.1 Every new board looks the same

One Linux image serves every board, so every board starts with the same identity: label `MPS3`, address `192.168.10.101/24`, MAC `02:00:00:4d:50:53`. Harness Manager says so:

```
identity    unset: ? · 192.168.10.101 · 02:00:00:4d:50:53; identity not set (default MAC): mac
02:00:00:4d:50:53 is the image default, which every board gets
```

At every boot the harness works out each field of its identity from the first of three sources: an override on the board's `/persist` (what `board identity` writes); values baked into the board's stage-0 (§9.6); the defaults above. The host name is the label in lower case unless you set one.

3.2 How a board joins a network

At boot the harness sets its MAC on the Ethernet interface before the link comes up, then:

- 1 **Asks for an address by DHCP**, two short tries, then goes on asking in the background. It sends its host name with the request, so a DHCP server that registers names in DNS can name the board.
- 2 **Adds its static address as well**, as a permanent second address, so the board is always findable on a direct cable or a private board network, even with a DHCP lease.
- 3 **Checks the static address first**. Before it adds the static address it asks whether anything else already answers on it (duplicate-address detection). If something does, it does not take the address, and logs `in use by another host ... not added`.

So a board on a network with a DHCP server has two addresses: its lease, which `pyverify identify` reports as `ip` with `"dhcp": true`, and its static address. A board with no DHCP server has only its static address. Stage-0's rescue mode uses only the static address.

3.3 Why each board needs its own MAC and address

Two boards left on the factory identity on one network break in ways that are hard to see:

- **The second board loses its static address.** Duplicate-address detection sees the first board on `192.168.10.101`, so the second never adds it. A direct-cable or rescue session to that address reaches the wrong board.
- **A shared MAC confuses the network.** A switch learns one MAC on one port at a time, so traffic for both boards goes to whichever spoke last. A DHCP server identifies clients by MAC, so both boards get the same lease, and only one of them can use it.
- **Tools cannot tell the boards apart.** Harness Manager pins each board's SSH host key by address; two boards answering one address look like a host-key change, and it refuses the connection.

Give every board a unique MAC, a unique static address and a unique label before it joins a shared network.

3.4 Giving each board its identity

The board needs the claim and `/persist` on its user microSD (chapter 1). Generate a locally administered MAC at random, so that it cannot clash with any manufacturer's address:

```
printf '02:%02x:%02x:%02x:%02x:%02x\n' $((RANDOM%256)) $((RANDOM%256)) $((RANDOM%256))
$((RANDOM%256)) $((RANDOM%256))
harness-manager board identity $B --label MPS3-LAB4 --mac 02:5a:91:3c:07:e2 --ip 192.168.10.104/24
```

- **The rules.** Label: A–Z, 0–9 and `-`, at most 16 characters so that it fits the panel's top row. IP: `a.b.c.d/nn`, a usable host address (`/24` if you give no prefix). MAC: unicast and non-zero. Anything else is refused before a byte is written.
- **What happens.** Harness Manager asks you to type the new label (`--consent` takes it on the command line), writes the override, restarts the harness, waits up to 180 s and reads the identity back. A board whose address changed stops answering at the old one: the result says the change was applied but not verified, and you reopen the board at the new address.
- **Undo.** `--unset ip` (or `label`, `mac`, `hostname`) drops one field; `--clear` drops the override, so the board falls back to its stage-0 values or the defaults.
- **No card, no override.** A board running without a user microSD has nowhere to keep the override (`no_persist`). It keeps its stage-0 values or the defaults; a per-board stage-0 is the alternative (§9.6).

`harness-manager board identity $B` with no change flags shows what the board says it is, field by field with its source, and warns about any clash with another board it knows.

3.5 Laying out a site

LAYOUT	HOW	SUITS
One PC per board	A direct cable from each board to its own PC (or its own port). The factory identity is fine; the PC uses <code>192.168.10.1/24</code> .	A desk, a teaching bench where each seat owns a board
A private board network	A switch for the boards and the PCs that use them, no DHCP server, one <code>/24</code> . Each board gets its own static address (<code>.101</code> , <code>.102</code> , ...) and MAC; PCs take addresses below <code>.100</code> .	A lab room; the simplest shared layout
The site network	The boards on a site network with DHCP. Give each board its MAC first, then a DHCP reservation for that MAC, so its lease never changes. Keep each static address unique as well, for rescue and direct-cable work.	Boards used from offices; needs the network team's agreement

Whatever the layout, keep a site table with one row per board: label, serial number, MAC, static address, DHCP reservation, the release it runs, and who to ask about it. It is what you will check first when two boards misbehave together.

CAUTION

The harness's network services have no passwords: anyone who can reach a board's control port can load a design, reset it, change its clock or restart the harness. Only SSH, JTAG, the logic-analyser port and changes to the card and the board's identity are protected, by the claim. Put boards on a network you trust, never on the open internet.

3.6 Finding a board, and telling boards apart

- **harness-manager probe** looks for boards on your PC's networks and on its USB ports, and says which serial port belongs to which board's MCC. **--host ADDR** tries an address you name.
- **harness-manager identify \$B** blinks the board's panel backlight for a few seconds with a banner naming you; a tap on the panel stops it. Use it before you touch a board's cables.
- **The front panel's top row** shows the board's label, and the status page shows its address and MAC (§8.4).
- **pyverify identify --host \$B** answers even while someone else holds the board's control port: mode, label, address, whether it is claimed, and the SSH host-key fingerprint.

3.7 Several people, one board

The claim decides who may log in, debug and change the card; it does not share the board out. The board has no lease of its own, so on a board several people can reach, one person's load or reset can interrupt another's session. On a board of your own, agree who is using it. Where sharing must be enforced, put the boards behind a hub with leases (chapter 6).

4 Health checks and soak tests

Three ways to ask whether a board keeps working, from the lightest: a quick check you run by hand, Harness Manager's overnight checks, and the soak tool that a release must pass. Run the quick check after any change to a board, and a soak before you trust a board to run unattended.

4.1 The quick check

Five minutes, any board, nothing changed except one load and a restore:

QUESTION	RUN	HEALTHY ANSWER
Is the harness answering, and is it the right one?	<code>harness-manager info \$B</code>	Control <code>idle</code> ; the expected <code>shell</code> ; <code>build firmware/fabric check: ok</code>
What is at this address at all?	<code>pyverify identify --host \$B</code>	"mode": "run", and <code>claimed</code> true in its <code>ssh</code> field
Has anything restarted by itself?	<code>pyverify stats --host \$B</code>	<code>os_up_ms</code> (Linux) and <code>up_ms</code> (the harness service) both match the board's last planned restart
Is Linux on the expected slot, confirmed?	<code>harness-manager slot status \$B</code>	The slot you committed, running and confirmed
Is the card healthy?	<code>harness-manager card status \$B</code>	Present; the kept design you expect (in v2.0.0 it does not load at power-on: §10.7)
Does a load work end to end?	<code>harness-manager program \$B led, then restore \$B</code>	Both <code>verified</code> ; about 35 s and 40 s

If `up_ms` went back to zero while `os_up_ms` kept counting, the harness service was restarted by Linux; if both did, the board rebooted. Either, unplanned, is worth a look at the harness log (`pyverify log --host $B`) before you carry on.

4.2 Harness Manager's overnight checks

The board's **Checks** tab runs every check a machine can judge through Harness Manager's own commands, round after round (every 30 minutes by default) until a set time, and writes a report per round and a `REPORT.md` per run. The run lives in Harness Manager's background service: you can close the window and it goes on.

NOTE

In Harness Manager 1.0.0, Checks runs only on a board behind a hub: it holds the hub's lease for the whole run so that nobody else changes the board under it, and on a board with no hub it refuses to start (`... is not behind a hub: the checks need its hub lease held here`). On a board of your own, use the quick check (§4.1) and the soak (§4.3).

- 1 Check the plan.** It is picked from the board: `linux` with a usable user microSD, `linux-netboot` with a blank one, `linux-nocard` with none. The line under it says why.
- 2 Choose what it may change.** `Read only`, or `Safe`, which also loads a design, puts `greybox` back and reads the MCC once per round. A run never writes a card, a slot or a config SD, never reboots, never claims and never forces anything.
- 3 Set the times:** `Run until` (default the next 08:30), `every` 15 to 60 minutes, and `Now` or `At` a start time.

- 4 **Settle the lease.** If you hold it, the service keeps it until the run ends; if the board is free, tick **Take the lease for the run**; if someone else holds it, **Start** is off and names them.
- 5 **Announce, then start.** Write the **announcement** fills in what the run will and will never do, for your message to the board's users.

Stop finishes the current check, puts `greybox` back and writes the report; **Past runs** keeps every run. The run uses the service's settings, not your shell's, so set the overlay directory once with `harness-manager config set mps3.overlay_dirs <dir>` before you start.

4.3 The soak

The soak tool asks whether a board keeps working unattended for a day or more, and is the test every release passes before it is published. It lives in the platform repository (<https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform>), `scripts/soak_linux/soak_linux.py`, and runs on a Linux PC that reaches the board's Ethernet and, for its power-on drills, holds the board's Debug USB. It needs Python 3.11 or newer and the platform's `pyverify` (set `PYVERIFY_DIR` to the checkout's `host/pyverify`).

MODE	WHAT IT DOES
accel	24 hours in three phases. Hours 0–6 and 18–24 are drills: every design loaded in turn, every DUT-clock preset, ten power-on boots by paced MCC reboot, harness reboots, watchdog trips and the OS-slot fallback. Hours 6–18 have no resets, so that leaks show. Throughout: a load every 3 minutes, an SSH login every minute, the board's counters every minute, a JTAG and a logic-analyser session every hour, and a resource sample every 5 minutes.
tail	A gentler 72-hour soak: a sample every 5 minutes, one load and one SSH login an hour, no drills.
boots	N power-on boots to SSH, on their own: how reliably the board comes up.
mcc-reboot	One paced MCC reboot, with the MCC's log captured.

- 1 **Try it without a board.** `--dry-run` runs the whole mode against a model of the harness on a compressed clock (24 hours in about 7 minutes); `--dry-run-inject hang@20` shows a stop rule firing.

```
python3 scripts/soak_linux/soak_linux.py accel --dry-run
```

- 2 **Set the board aside.** The drills reboot and power-cycle it: tell its users, and close it in any Harness Manager, which would hold its control port.
- 3 **Run it.** Name the board, the shell it must report, the designs to cycle through, the SSH key it logs in with, and the MCC's serial port:

```
python3 scripts/soak_linux/soak_linux.py accel --host $B --expect-static-id 0x44EE76D5 \
--overlay ~/mps3-bundle/overlays/led --overlay ~/mps3-bundle/overlays/uart_echo \
--ssh-opt=-i --ssh-opt=$HOME/.ssh/id_ed25519 --mcc-tty /dev/ttyUSB0 --ssh-p99-max 20 --
out soak.jsonl
```

- **What stops it.** Each of these is a FAIL, with forensics collected: the control port offline or wedged, a wrong `static_id`, a reset nobody asked for, the harness service restarting, a load that did not verify, a failed SSH login, rescue, a kernel oops, or a failed drill or debug session. At the end it checks that memory and file handles stayed flat and load times stable through the quiet phase.
- **The verdict** is the exit code: 0 PASS, 1 FAIL, 2 a usage or set-up error, 3 INCONCLUSIVE (stopped early, or a criterion it could not judge).

- **The bar a release meets** is a PASS over the full 24 hours, with the slowest 1 % of SSH logins taking 20 s or less.

Behind a hub, the soak runs on the hub, under the group that may open the board's serial ports, and extends the lease with `--heartbeat-cmd`; chapter 6 covers the hub's side.

5 The recovery ladder

Most boards that look dead are not. This chapter is the order in which to bring a board back, from the step that changes least to the step that needs a JTAG cable. Ask the board first, then go down the ladder one rung at a time, and stop at the first rung that works.

5.1 Ask the board first

`harness-manager info $B` reports the harness's state, with advice:

STATE	MEANS	GO TO
idle	Ready. The fault is in the design, or in your PC	Rung 1
busy	Another program holds the board (a window left open, a console, a script), or the harness is finishing a failed load	Close the other program, or wait 30 s
wedged	The harness took the connection and never replied	Rung 2
rescue	Stage-0 found no Linux to boot, and waits for one over the network	§5.3
offline	Nothing answers. Check power, the cable and your PC's address first	<code>pyverify identify</code> , then rung 3

`pyverify identify --host $B` answers even when the control port is held: `"mode": "run"` is Linux; `"rescue"` is stage-0, with a reason; `"nohw"` is Linux with no harness hardware to drive, so the config SD holds a different shell from the one this Linux was built for; silence means nothing on the network side is running.

5.2 The ladder

- 1 Restore greybox, then reset the DUT** (`harness-manager restore $B`, `harness-manager reset $B`). This clears a misbehaving design while the harness answers.
- 2 Restart the harness.** In the window, Board → Recover → Restart the shell; or `pyverify reboot --host $B --wait`. Linux boots again without reloading the FPGA, in 3 to 4 minutes. It drops every connection to the board.
- 3 Reload the FPGA from the config SD** with the Debug USB: Board → Recover → Reboot via the MCC, or `harness-manager mcc - --serial <port> reboot`. Harness Manager paces the MCC's command and proves the board went down and came back (§5.6).
- 4 Press USER_nPB0**, the harness's reset button, if a power-up left the board stuck: no SSH, and `identify` silent or in rescue with `ddr calib fail`. It restarts the harness with the board's clocks already running.
- 5 Power-cycle:** off for ten seconds, then wait 5 minutes before you judge. With a networked power plug configured in Harness Manager, `harness-manager power cycle $B` does it from your PC.
- 6 Push Linux from your PC** if the board sits in rescue (§5.3).

- 7 Put the config SD back.** Restore the backup that `sd install` made, reboot as in rung 3, then install the release again (§1.3). If the `V2M-MPS3` drive does not appear at all, power off, take out the config microSD, unzip the backup onto its root on your PC, and put it back.

```
harness-manager sd - --volume /media/$USER/V2M-MPS3 restore ~/mps3-backups/<zip>
```

- 8 Use a JTAG cable on J17:** read the FPGA's configuration state (§5.4), and load the harness over JTAG to bring the board back now (§5.5).

After any restart, reload, power-cycle or recovery, load `greybox` once: the partition stays in reset until the first load, so a design will look dead until then.

CAUTION

Never halt the harness's processor (the MicroBlaze V) over JTAG. That stops Linux, and the watchdog then resets the board about 43 s later. Read the harness's state over the network instead: `pyverify log`, `pyverify mailbox` over SSH, or `dmesg` through `harness-manager board ssh $B`.

5.3 Rescue, and Linux from your PC

When stage-0 has no Linux to boot, it waits in **rescue**: it answers ping, `identify` and TFTP, and nothing else. Its reason says why:

REASON	DO
no card	Insert the user microSD, or push Linux from your PC (below) to work without one until the next reset.
card has no stage0 slots, no valid slot	The card has no bootable Linux: write it again (§1.4), or push Linux and then push it into a slot.
slots exhausted (unconfirmed boots)	Both slots were tried and never confirmed a healthy boot. Push Linux from your PC, then look at the card (<code>slot status</code> , <code>slot verify</code>) and push a good image into a slot.
ddr calib fail	The board's memory did not start. Every push is refused. Press <code>USER_nPB0</code> ; if it persists, power off and reseal the DDR4 module.

```
pyverify netboot --status --host $B # what stage-0 is doing, and why
pyverify netboot ~/mps3-bundle/linux_slot.img --host $B \
--tool <checkout>/src/linux_soc/hw/fw_stage0/stage0_push.py
```

The image boots once, from memory, in about a minute; nothing is written to the card. Its exit code: 0 accepted; 1 a bad image file; 2 refused by stage-0 (`--status` says why); 3 no stage-0 at that address; 4 the transfer failed. Linux running from memory comes up unclaimed, with a new host key, so claim it again before you repair the card (§1.4, the no-reader route, from step 2).

5.4 A dark board: read the FPGA over JTAG

"Dark" (no answer to ping or `identify`) has two causes that look the same from the network, and only the FPGA's configuration registers tell them apart. The MPS3 has no FPGA JTAG on its Debug USB, so this needs a JTAG cable on header J17 and AMD Vivado's hardware manager on the PC it is plugged into. Reading the registers halts nothing and works on a board that holds nothing at all. In Vivado's Tcl console:


```

open_hw_manager
connect_hw_server -url localhost:3121
source scripts/mps3_hw_target.tcl          # from the platform repository
set d [mps3_open_hw_target {*<cable-serial>*}] # exactly one board, by its cable's serial
refresh_hw_device -quiet $d
foreach p {REGISTER.CONFIG_STATUS.SLR0.BIT[14]_DONE_PIN
           REGISTER.CONFIG_STATUS.SLR0.BIT[04]_END_OF_STARTUP_(EOS)_STATUS
           REGISTER.CONFIG_STATUS.SLR0.BIT[00]_CRC_ERROR
           REGISTER.USERCODE.SLR0} { puts "$p = [get_property $p $d]" }

```

READING	MEANS	NEXT
<code>DONE = 0</code> , with or without <code>CRC_ERROR</code>	The FPGA holds nothing: the MCC never loaded the config SD, or loaded a damaged file	Suspect the config SD. Load the harness over JTAG to restore service (§5.5), then restore or reinstall the config SD (rung 7).
<code>DONE = 1</code> , <code>EOS = 1</code> , no CRC error, the expected <code>USERCODE</code>	The FPGA took the right base cleanly; the fault is later	Read the harness serial console (§5.7) for stage-0's lines. Then load the same file over JTAG: if the board now runs, suspect the power-up sequence and use <code>USER_nPB0</code> ; if not, the file itself is bad.
Another <code>USERCODE</code>	The config SD holds a different build from the one you think	Check your record and the card before anything else.

The KU115's property names carry the SLR (`SLR0`); the names without it do not exist. Keep the brackets inside braces, as shown, or Tcl reads them as commands. A JTAG port that answers at all means the board has power. An Ethernet link light says nothing about the FPGA: the PHY links even when the FPGA holds nothing.

5.5 Loading the harness over JTAG

`scripts/mps3_recover.sh` in the platform repository (<https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform>) loads a full base bitstream into the FPGA over the J17 cable, in about 30 s, with Vivado found on the path or in `$VIVADO`:

```

MPS3_JTAG_CABLE='*<cable-serial>*' scripts/mps3_recover.sh <release>/config-
sd/MB/HBI0309C/Nanosoc/nanosoc.bit

```

- **Always pass the bitstream:** the one on the board's config SD. The script's built-in default is not a release file.
- **It is volatile.** The next power-up or MCC reboot loads the config SD again. It restores service; it fixes nothing on the card.
- **It refuses to guess.** It opens only a target that matches the cable filter, refuses unless exactly one board matches, and trusts only its own `shell loaded` line. Exit codes: 1 bitstream not found, 3 no Vivado, 4 Vivado failed, 5 Vivado exited 0 without loading.

5.6 The MCC: pacing, and what never to send

The MCC loads the config SD only at power-on and on its `REBOOT` command. It drops characters it receives in a burst, so a reboot must be paced: the MCC's own console port only (the first of the Debug USB's four serial ports; the other three are FPGA UARTs, where `REBOOT` does nothing), one reader on that port, a bare carriage return first, then `REBOOT` one character at a time (Harness Manager waits 60 ms a character over the Debug USB and 100 ms through a hub's serial share). Harness Manager's `mcc ... reboot` does this and reports what the MCC loaded. Close any other program on that port first: on Linux, Harness Manager cannot always tell that another reader has it open. Confirm the result from the board (`info` , `identify`), never from the tool's "ok" alone.

Never send the MCC `FORMAT`, `DEL`, `EEPROM`, `USB_OFF` or `SHUTDOWN`; Harness Manager refuses them (exit 15). Never update the MCC's firmware to fix a platform problem (§1.2). And never mount the config SD while the board boots: while your PC holds it, the MCC cannot read it, and the board comes up dark.

5.7 The harness serial console, and releasing a claim

The harness prints stage-0's messages, the kernel's and its own on the **harness serial console**: the third of the Debug USB's four serial ports (FPGA UART lane 2), at 115 200 baud. It is also the only place the board's root password works. On Linux it is usually `/dev/ttyUSB2`:

```
screen /dev/ttyUSB2 115200
```

A claim cannot be released over the network. If the key that claimed a board is lost, log in as root on the serial console and run `mps3-unclaim`: it lists the keys, asks, removes the claim and reboots the board, which comes back unclaimed. Then claim it again (§1.5).

6 Running a hub for a group

Everything so far works with no infrastructure. When a group shares boards, a **hub** adds what a board lacks: a queue of leases, so that one person's load cannot interrupt another's, and remote access to each board's Debug USB, so that its config SD and MCC can be reached with nobody in the room. This chapter is the site team's side of a hub; the user's side is *User Guide* ch 10.

6.1 What a hub is

A hub is a Linux machine that holds each board's Debug USB and gives each board a private Ethernet link. It runs **fpgahub**, an open-source board-sharing service, which names each board's devices, keeps the lease queue, writes config SDs, reboots MCCs and shares serial consoles. Users reach the boards through SSH to the hub; the hub is not in the data path of a session.

- **Per board, the hub holds** the Debug USB (its four serial ports appear as `/dev/<target>/tty_00` to `tty_03`: `tty_00` is the MCC, `tty_02` the harness serial console; the config SD appears as a USB drive), a USB Ethernet adapter on a private `/24` of the board's own, and, optionally, a JTAG cable on J17.
- **Each board needs its own subnet and identity** on a hub, since each sits on its own link: give it its static address and MAC as chapter 3 describes, with the hub's adapter at `.1` on the same `/24`.
- **UDP stays on the hub.** Every TCP port a user needs crosses the SSH hop, but UDP does not, so `identify`, the claim, and a Linux push from rescue run on the hub itself.
- **fpgahub cannot power-cycle an MPS3.** Add a networked power plug if you need that.

6.2 Setting up the hub

fpgahub's own documentation covers installing it and each board type, with an MPS3 section. In outline, on a Linux host with Python 3.11:

```
sudo ./packaging/install.sh --role server          # the fpga group, the fpgahubd service,
config.toml
sudo systemctl enable --now fpgahubd
fpgahub discover                                # the USB hub paths
fpgahub target add <target> --server <hub> --hub <usb-path> \
--host-ip 192.168.N.1/24 --board-ip 192.168.N.101 --board-mac 00:00:00:00:00:00
fpgahub apply --role server                      # udev names
fpgahub network apply                           # the board's link
```

Three things differ for an MPS3:

- **The MCC is interface 00**, so `tty_00`. Configure its reset by interface number, never by a pinned `tty_path`: a stale path wins over the number, and a reboot sent to an FPGA UART reports success and does nothing.
- **The hub path is the board's own USB hub**, the one that holds the config SD. The Ethernet adapter hangs one level deeper on another branch. If fpgahub finds `--board-mac` on the bus it pins the board to the adapter's path and then cannot find the config SD, so give the all-zero MAC shown above. For the same reason never run `fpgahub config reconcile-macs --apply` on an MPS3 record.
- **Leave the lease gates off.** fpgahub can restrict a board's serial ports and Ethernet to the lease holder; for the MPS3 both stay off, so leases are advisory (§6.4).

Give each user an account on the hub, an SSH key that logs in without a prompt, and membership of the `fpga` group. Opening fpgahub's socket is its authorisation: every caller on the socket is an administrator. A non-interactive SSH session does not pick up a group added after its credentials were made, so run hub commands under the group, `sg fpga -c "fpgahub ..."`, never under `sudo`, which acts as another holder and ends

someone's lease. Harness Manager does this for its users when they add the hub with `harness-manager hub add <name> --ssh <hub>`.

6.3 The claim on a shared board

Claim each shared board with a keys file that holds the site team's keys, the hub's own key for its automated jobs, and the key of each user who may debug (one public key per line, at most 64). Users never claim a shared board: Harness Manager *adopts* the existing claim with a key that is in it (`board claim --adopt`), which adds nothing. Claim from the hub, since the claim travels over UDP, and only once `identify` says `"mode": "run"`:

```
hub$ pyverify claim --host <board-ip> --key <group-keys.pub>
```

A second claim is refused, so to add or remove a user's key, log in with a team key, edit `/persist/ssh/authorized_keys`, then run `/usr/sbin/mps3-keys-sync` (not yet rehearsed in this release: keep a team key in the file, and the serial console within reach, before you edit it), which rebuilds the key file SSH reads. To release the claim, share the board's harness serial console from the hub (`fpgahub share start <target> /dev/<target>/tty_02 --baud 115200`) and run `mps3-unclaim` as root (§5.7). To make a board's identity match its hub record, `harness-manager board identity <board> --from-hub`.

6.4 Leases

- **The unit is the board**, and the holder is the hub account that asked. Every session of one account is the same holder: a second acquire refreshes the first lease. Tools that check the holder (`pyverify sd field`, `boot-rate`) need the name `fpgahub lease show` prints.
- **Time**. A lease lasts an hour by default and runs out unless renewed; Harness Manager renews it while the board is open. `--tier background` marks a long unattended run that any interactive request displaces.
- **What a lease stops**. A dispatch through the hub (program, reset, debug) on a board someone else holds is refused, except for administrators. Nothing on the board checks a lease: **leases are advisory**. Agree house rules with your users (*User Guide* ch 10) and keep them.
- **Requests and force-release**. A user asks for a held board from Harness Manager; the holder can release it or keep it for 5 to 60 more minutes; with no answer in two minutes, the first in the queue may force it, which stops whatever the holder was running. An administrator can end any lease from the hub:

```
sg fpga -c 'fpgahub board lease revoke <board> --reason "stuck overnight run" --yes'
sg fpga -c 'fpgahub target lease-history <target>'
```

6.5 Config-SD writes and MCC reboots through the hub

With a hub, a release can be installed on a board with nobody in the room. `pyverify sd field` does the whole job and refuses at the first doubt: it checks the lease holder and the file's hash, makes **one** config-SD write through fpgahub, waits for fpgahub's journal to confirm it, and only then sends the paced MCC reboot and reads the MCC's log to `FPGA configuration complete`. Run it on the hub, after a dry run:

```
pyverify sd field <base.bit> --target <target> --holder <you@hub> \
--expect-sha256 <12 or more hex digits of the file's sha256> --dry-run
```

Its exit codes: 0 done; 1 refused before the write; 2 usage; 3 hub unreachable; 4 no journal confirmation (no reboot sent); 5 the reboot was refused, unacknowledged or did not configure. After a 5, retry only the reboot, with `--already-written`.

CAUTION

A config-SD write through fpgahub always reports `timed out`: writing about 12 MB over the card's USB link outlasts the client's 30 s limit, while the hub goes on writing for 70 to 80 s. Never retry, reboot or mount the card mid-write: a second write or a reboot over the first corrupts the card, and the board comes up dark. Wait for fpgahub's journal line (`program dispatched: ... method=sd ... ok=True ... sha256=...`), or 5 minutes.

For a reboot with no write, Harness Manager's `mcc <board> reboot` runs the paced reboot on the hub (not yet run on a board in this release). The rules of §5.6 apply on the hub too: `tty_00` only, one reader (a console share on `tty_00` steals the reply), a bare carriage return first, 100 ms per character. Use only `--method sd` for fpgahub's own `target program`, and do not rely on its `target reset --method mcc`, which sends the command in a burst.

6.6 Checks, soaks and boot rates on a hub

Harness Manager's overnight checks (§4.2) need a hub, and hold its lease for the run. The soak (§4.3) runs on the hub under `sg fpga`, with `--mcc-tty /dev/<target>/tty_00` and a `--heartbeat-cmd` that renews your lease. `pyverify boot-rate --method mcc` reboots a board N times and records how often it comes up, with a 95 % interval; it needs a lease held by `--holder` and never takes one.

6.7 Hub traps

- **Never mount a board's config SD by hand on the hub.** While the hub holds it the MCC cannot read it; fpgahub mounts and unmounts it for its own writes.
- `fpgahub share stop <target>` stops every share on the target, other people's too. Agree it first.
- **Stage files for fpgahub outside `/tmp`:** the service has a private `/tmp` and cannot see yours.
- **Many short SSH connections from one account** can hit the hub's limit on unauthenticated connections. Reuse one connection for scripted hub calls (`ControlMaster`), but never for a forward to a claimed board, which must use its own connection.

PART 2 · HOW THE HARNESS WORKS

7 Hardware and technologies

The harness is built from a few well-established technologies; the value is in how they are combined. This chapter explains each just far enough for the rest of Part 2, then describes the board: its FPGA, how it boots, its clocks, and which interfaces belong to the harness and which to the design. Arm's *MPS3 Getting Started Guide* (107789) and *MPS3 Technical Reference Manual* (100765) remain the authority for the board.

7.1 Dynamic Function eXchange

An FPGA is configured by writing a *bitstream* into its configuration memory, normally the whole device at once, with everything on it stopped. **Dynamic Function eXchange** (DFX, AMD's partial reconfiguration) splits a design into a **static region**, configured once, and **reconfigurable partitions**, each rewritten on its own by a *partial bitstream* while the static region keeps running. A design for a partition is a **reconfigurable module** (RM); this guide calls the packaged form an **overlay**.

The static region and every RM are placed and routed together once, in one multi-configuration build, which *locks* the static's placement and routing so that every RM meets the partition on the same wires, the **partition pins**. The platform identifies a locked static by a 32-bit fingerprint, the `static_id` (§10.5). A partial is valid only on the static it was built against.

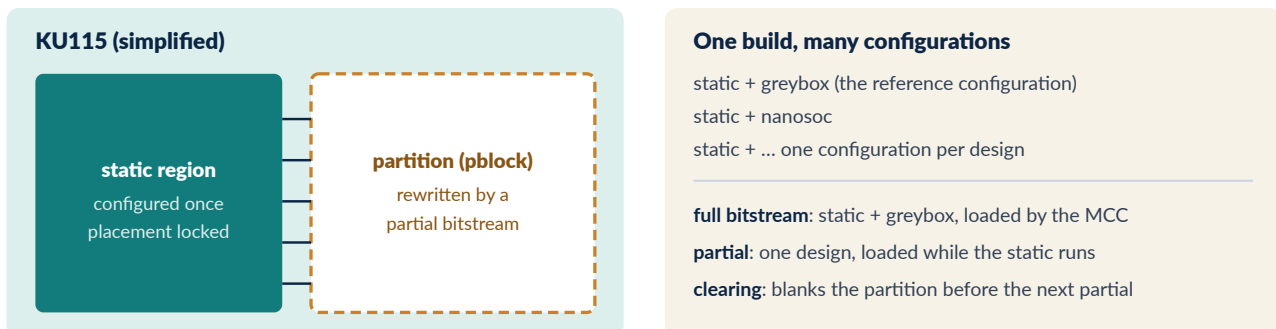


Figure 7.1. The DFX split. The static region and every design are implemented together once; afterwards each design exists as a partial bitstream that fits only the static it was built with.

Clearing bitstreams. The MPS3's FPGA is a Kintex **UltraScale** part. On UltraScale, unlike UltraScale+, a partition must be *cleared* before a new partial goes in, using the clearing bitstream of the design currently loaded. That is why every overlay carries a clearing file beside its partial, and why the harness must always know what is loaded (§10.3). While a partition is rewritten its outputs are undefined, so a **decoupler** holds every signal leaving it at a safe value (§10.4).

7.2 The configuration port, the processor and the debug links

- **ICAP.** The Internal Configuration Access Port (`ICAPE3`) lets logic inside the FPGA write configuration frames. AMD's AXI HWICAP puts a FIFO and registers in front of it, so the harness's processor streams a partial word by word as it arrives over Ethernet.
- **A soft processor running Linux.** The FPGA has no hard processor, so the harness runs on an AMD **MicroBlaze V**: a RISC-V core (`rv32imac` with the Zba, Zbb and Zbs extensions) with an Sv32 memory-management unit, at 100 MHz, with 1 GiB of the board's DDR4. The MMU lets it run Linux 6.18 (started by

OpenSBI 1.6, built with Buildroot 2026.02), so the harness's services run as ordinary processes with memory protection, remote login and a normal network stack.

- **JTAG and CoreSight.** JTAG (IEEE 1149.1) is a four-wire serial protocol for reaching test and debug logic inside chips. A Cortex-M processor exposes its debug through CoreSight: a *debug port* that speaks JTAG or Arm's two-wire SWD, and *access ports* that turn debug requests into bus transactions. The nanoSoC uses the CoreSight SoC-400 SWJ-DP and an AHB-AP that can read and write the whole memory map, halt the core and set breakpoints. The harness drives the design's JTAG pins itself and serves them over the network (chapter 11).
- **XVC and logic analysers.** An Integrated Logic Analyzer (ILA) is FPGA logic that samples internal signals when a trigger fires. Vivado normally reaches ILAs over a JTAG cable; **Xilinx Virtual Cable (XVC)** carries the same JTAG shifts over TCP, conventionally port 2542. The shell serves XVC into a debug bridge, so ILAs placed inside a design are reachable from Vivado anywhere (§11.5).
- **Ethernet.** The harness reaches the network through the board's SMSC/Microchip **LAN9220**, a 10/100 controller with MAC and PHY in one package on a 16-bit asynchronous bus, driven by Linux's standard `smc911x` driver. A design's own Ethernet MAC speaks RMII to a *virtual PHY* inside the shell (§8.3); it has no physical port.

7.3 The board and its FPGA

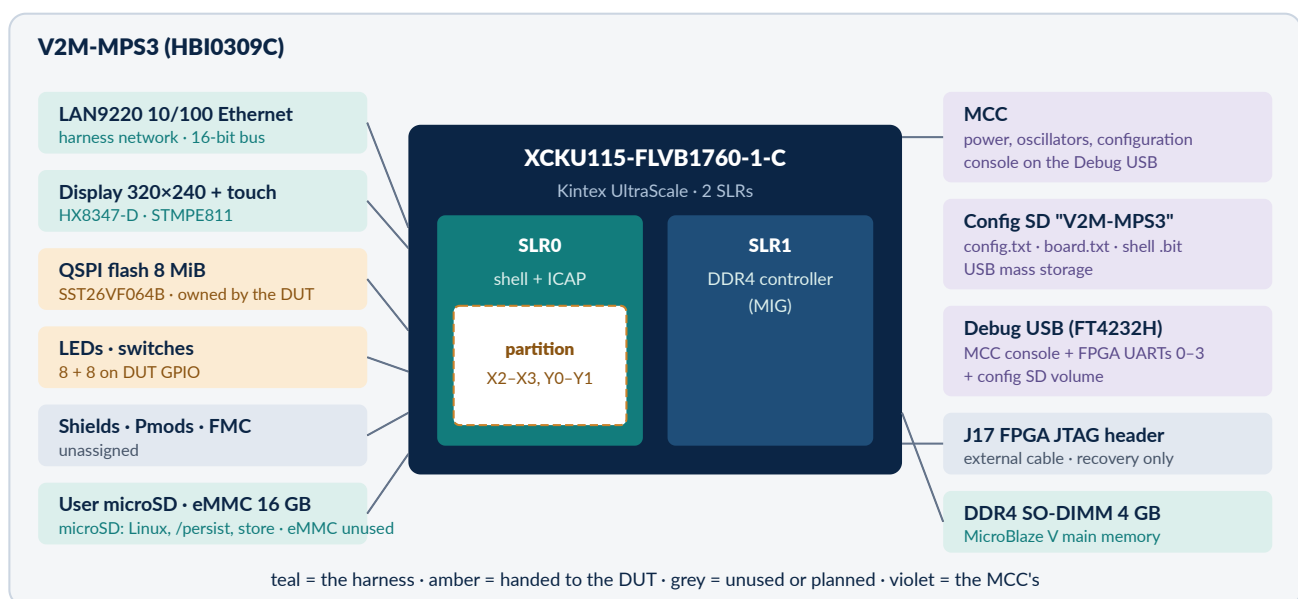


Figure 7.2. What the platform uses on the MPS3. The MCC and its config SD configure the FPGA; after that, everything reaches the board through the LAN9220.

The FPGA is an **XCKU115-FLVB1760-1-C**, built from two super logic regions (SLRs). **SLR0** holds the static shell, the ICAP and the partition: clock regions X2Y0, X3Y0, X2Y1 and X3Y1, which give 42 824 LUTs and 144 block-RAM tiles; the single-core nanoSoC uses about a fifth of those LUTs. **SLR1** holds the DDR4 memory controller (the MIG), in clock regions X2Y5–X2Y7 beside I/O banks 49–51, outside the partition.

7.4 How the board boots

The MPS3's **Motherboard Configuration Controller (MCC)** is a microcontroller that owns power, the oscillators and FPGA configuration, and can load only a *full* bitstream. At power-on, and on a `REBOOT` command at its console, it reads the **config SD**, a FAT volume labelled `V2M-MPS3` that the Debug USB presents as a USB drive, in this order:

- `config.txt` at the root (`AUTORUN: TRUE`, `USB_REMOTE: TRUE`, `UARTMODE: 0` and others);
- `MB/HBI0309C/board.txt` (or `HBI0309B` on a revision B board), whose `APPFILE:` line names the application file. Its path must use a DOS backslash: a forward slash leaves the FPGA unprogrammed;
- the application file, whose `F0FILE:` names the base bitstream, and which sets the oscillators.

The base bitstream holds the static shell with `greybox` in the partition and the boot loader, **stage-0**, in the processor's local memory. When configuration ends, stage-0 starts and brings up Linux (§9.2). So changing the shell or stage-0 means rewriting the config SD and rebooting the MCC; changing Linux does not.

Cold start. After it configures the FPGA, the MCC goes on programming the oscillators, among them the DDR4 reference. A stage-0 that starts calibrating the memory inside that window can fail or stall, so the release's stage-0 waits for the clocks to settle after a power-up, and arms the watchdog from its first instruction so that a stall recovers by itself. Warm resets (`USER_nPB0`, a harness restart, the watchdog) never had the fault. Because stage-0 lives in the base bitstream, only a config-SD write brings a stage-0 fix to a board.

7.5 Clocks

OSCILLATOR	FREQUENCY	USE
OSC0	25.0 MHz	The LAN9220's PHY reference. It must be exactly 25.0 MHz: 24.0 breaks Ethernet.
OSC1	50 MHz	The shell's clock input (pad AK16), multiplied to 100 MHz for the processor, the AXI interconnect and the ICAP, and to a fixed 50 MHz RMI reference.
OSC6	100 MHz	The DDR4 reference, programmed by the MCC after configuration. The memory controller runs at 200 MHz.
DUT clock	50 MHz	From the shell's <code>clk_wiz_dut</code> . The harness can retune it to a 25, 50 or 100 MHz preset (§8.2), and the soak test does, but v2.0 supports only 50 MHz: the designs are timed and their consoles paced for it, so Harness Manager refuses a change.

7.6 Who owns each interface

INTERFACE	OWNER	NOTES
LAN9220 Ethernet	Harness	The kernel's <code>eth0</code> . DHCP first, plus the board's static address (§3.2).
Display and touch	Harness, lendable	320×240, Himax HX8347-D on an 8-bit 8080 bus; STMPE811 touch on I ² C. A design can borrow the panel (§8.4).
QSPI flash	Design	SST26VF064B, 8 MiB, for the design's flash boot. It powers up write-protected, and the harness never writes it.
LEDs, DIP switches	Design	Eight LEDs on design GPIO 7:0; eight switches on GPIO 15:8.
Push buttons	Harness	<code>USER_nPB0</code> is the shell's power-on reset (and the DDR4 controller's). <code>USER_nPB1</code> turns the panel's pages and hands the panel to the design; held at power-on, it skips the design kept on the card (§10.7).
FPGA UARTs	Harness	Lane 2, wired to the Debug USB's third port at 115 200 baud, is the harness serial console (§5.7). Lane 1 sits behind a mux the MCC drives; lane 0 belongs to the MCC.
DDR4 SO-DIMM	Harness	The processor's main memory: a 1 GiB window at <code>0x8000_0000</code> of a 4 GB module, at 1600 MT/s through the MIG in SLR1.
User microSD	Harness	FPGA-owned, separate from the config SD; SPI mode, owned by the kernel. It holds the OS slots, <code>/persist</code> and the design store (§9.3).
Shields, Pmods, FMC	Unassigned	Two Arduino-style headers (36 I/O at 3.3 V), with Pmods J28, J34 and J38 on the same nets. No shell pin reaches them.
eMMC	Unused	16 GB, FPGA-owned.

INTERFACE	OWNER	NOTES
J17 JTAG header	Site team	The MPS3 has no FPGA JTAG on its Debug USB. An external cable on J17 is for recovery only (§5.4).

No power or current sensor is reachable from the FPGA, and there is no data path between the FPGA and the MCC. So the harness never reports power and cannot command the MCC: board reboots and oscillator changes go through the Debug USB. Appendix A lists every pin the shell constrains.

8 The static shell

The static shell is a small system-on-chip: a processor, a network interface, a configuration engine, debug bridges, a display, and the logic that fences off the partition. It is one Vivado block design, `fpga/shell/bd/shell_bd.tcl`, instantiated beside the partition in `shell_top.sv`. The block design has one processor seam, `SHELL_CPU: mbv` builds this release's harness (`cpu_mbv.tcl`: MicroBlaze V, DDR4 and the user microSD) and `mb` the older bare-metal shell (Appendix C). Every other block is the same Tcl at the same address in both.

8.1 The blocks

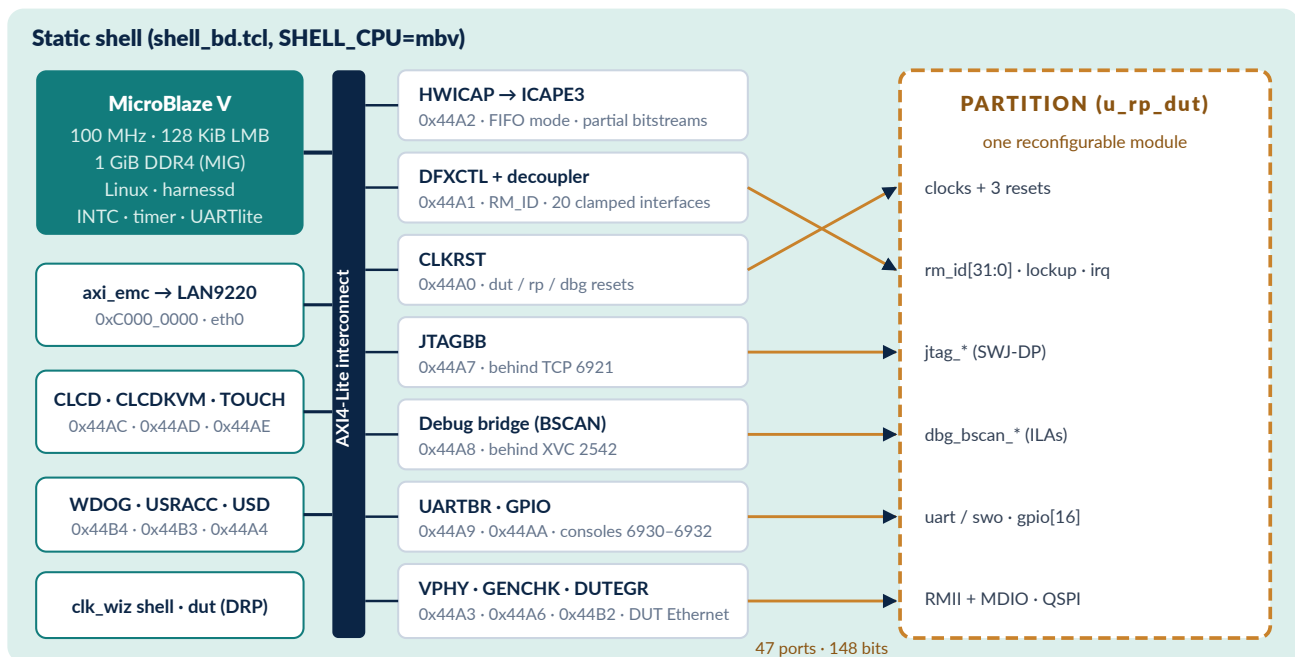


Figure 8.1. The static shell of the Linux harness. Every block the host depends on is on the static side of the boundary; the partition sees only scalar signals and byte streams.

Each block has a 64 KiB window in the `0x44A0_0000 – 0x44B4_FFFF` region; Appendix B lists them all with their owners. Under Linux each block has exactly one owner: the kernel, through a driver, or the harness service, through a userspace I/O mapping (chapter 9). The register map is generated from the block design (§13.3), and its contract, `docs/contracts/shell-regmap.md`, is the authority for every bit.

8.2 The registers that matter most

REGISTER	WHAT IT DOES
<code>CLKRST.RESET_CTRL</code> <code>0x44A0_0000</code>	Bit 0 releases the design's reset (<code>dut_resetrn</code>), bit 1 the partition's (<code>rp_resetrn</code>), bit 2 the debug reset. A swap holds the partition in reset throughout; after any harness restart it stays in reset until the next load.
<code>CLKRST.STATUS</code> +0x0C	Bit 0 the DUT clock's MMCM is locked; bit 1 the DUT clock is alive.
<code>DFXCTL.DECOUPLE</code> <code>0x44A1_0000</code>	1 isolates the partition (§10.4). Only a board power-on clears it; a watchdog reset <i>sets</i> it; a write of 0 releases it.
<code>DFXCTL.RM_ID</code> +0x10, <code>RM_STATUS</code> +0x14	The design's <code>rm_id</code> , read live from the partition pins, and its valid bit; bit 1 is the design's lockup, bit 2 its Ethernet interrupt. A load is verified by reading these back.

REGISTER	WHAT IT DOES
<code>MMCM_DRP</code> 0x44AB_0000	The DUT clock's MMCM, reprogrammed at run time: from 50 MHz in, with the VCO at 1000 MHz, presets of 25 MHz (O = 40), 50 MHz (O = 20, the default) and 100 MHz (O = 10).
<code>TELEM.STATUS</code> 0x44A5_0010	Bit 0: the DDR4 calibrated. There is no power sensor; the other telemetry registers read 0.
<code>USRACC.VALUE</code> 0x44B3_0004	The bitstream's <code>USR_ACCESS</code> stamp: which harness version the fabric is (§10.5).
<code>WDOG</code> 0x44B4_0000	The shell's watchdog: it resets the processor 21.5 to 43 s after the last kick (§9.2).

8.3 The design's Ethernet bench

The shell has a complete bench for a design's own Ethernet MAC, with no package pins: a **virtual PHY** (VPHY: the 50 MHz RMII reference, the design's receive pins, its transmit pins sampled, and MDIO from a register model), a **generator and checker** (GENCHK: traffic, injected faults and counts, and the design's IP address sniffed from its traffic), and an **egress capture** block (DUTEGR: up to 16 frames the design sent) behind a three-port bridge. *User Guide* ch 5 is the user's side.

8.4 The front panel

The shell drives the 320×240 panel as a text display of 40 columns and 15 rows, with two pages. The harness service renders it in the colours Harness Manager uses: labels grey, values white, healthy words green, warnings amber, and white-on-red only for faults.

STATUS PAGE ROW	SHOWS
0	The board's label (§3.4); on the right, the lease badge that Harness Manager relays for a board behind a hub (the board itself cannot see a hub)
2-9	<code>design</code> the loaded design, its version and <code>✓verified</code> ; <code>prog</code> the last load; <code>shell</code> the <code>static_id</code> and the card; <code>net</code> the address and link; <code>up</code> the uptime; <code>dut</code> reset, clock and MMCM lock; <code>icap</code> the configuration counters; <code>cfg</code> the design's description
10-12	A banner, when there is one: a fault first, then Identify, then the design's message, then load progress
11	With no banner: Harness Manager's session (<code>hm</code> , the user and their machine)
12	With no banner: <code>sys linux ssh claimed SHA256:...</code> , the claim and the first characters of the claiming key's fingerprint
14	<code>mac</code> the board's MAC and a heartbeat. A tap here, or a short press of <code>USER_nPB1</code> , turns the page

- **During a load** the status page shows `design partition decoupled` , `prog pushing led 55%` with a progress bar on row 10, `dut rst-hold` , and `cfg unknown: no valid design id` .
- **Identify** (§3.6) blinks the backlight at 2 Hz and turns rows 10–12 into a blue banner naming who asked, with `tap here to say you found it` .
- **The apps & ports page** lists, for the board's address, the command for each service: `ctrl` (6900), `tftp` (69), `push` (6910), `xvc` (2542) and `ssh root@<ip>` , with the footer `PB1 tap: next page hold: give DUT` .
- **The glass never lies**. The design row is read from `DFXCTL.RM_ID` on every refresh, never from a cache. In v2.0.0 a design the harness has no name for shows as `rm?<id>` (for example `rm?01000005` for `nanosoc_upy`); v2.0.1 names them.
- **A design can borrow the panel**. The upper eight design GPIO bits carry an 8080 bus through the **display tunnel** (`docs/contracts/dut-display-tunnel.md`). The CLCDKVM block hands the panel over on a long press of `USER_nPB1` or a command, and takes it back whenever the partition is decoupled or in reset.

The display driver sends a bounded number of bytes per pass and drops pixels rather than stall the network, so a busy board may repaint slowly; it never misreports.

9 The harness Linux

The harness's software is a small Linux system on the MicroBlaze V: a boot loader baked into the bitstream, Linux from the user microSD, and one service, `mps3-harnessd`, that answers the network. This chapter follows it from power-on to a running service, then describes the card, the services, the claim, the board's identity and the network protocol.

9.1 The layers

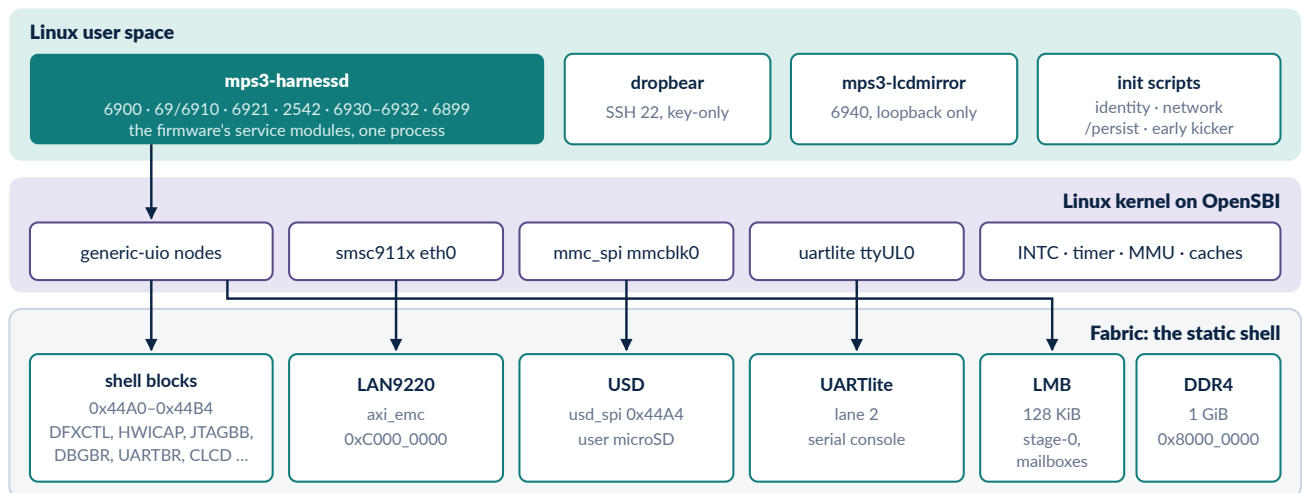
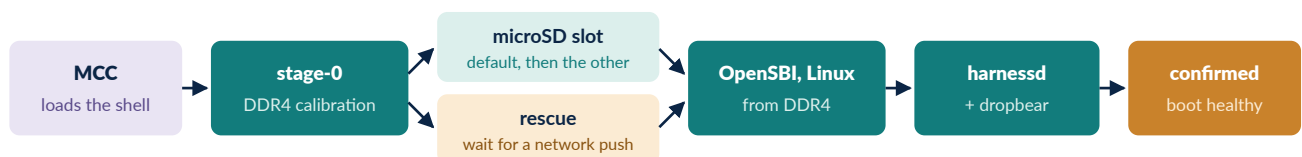


Figure 9.1. The Linux harness in layers. Each fabric block has one owner: the kernel (a driver binds it) or `harnessd` (through a `generic-uis` node, mapped by physical address), never both. `harnessd` also reads the top page of the local memory through UIO.

- **The kernel owns** the network controller, the user microSD (through `mmc_spi`), the console UART, the interrupt controller, the timer and the DDR4.
- **`mps3-harnessd` owns** every other block, mapped by physical address through `generic-uis` nodes. The owners are declared once, in `tools/gen_regmap.py`, which refuses a block without one; the device tree follows from it.
- **Around it**, `dropbear` serves SSH on port 22 (key-only), `mps3-lcdmirror` serves a copy of the panel to the board itself for Harness Manager's live display, and `mps3-debug` starts OpenOCD on demand (chapter 11). Start-up scripts resolve the board's identity, bring up the network (§3.2), mount `/persist` and feed the watchdog until `harnessd` runs.

9.2 How it boots



A watchdog armed by stage-0 resets the processor if `harnessd` is not kicking it within 42.9 s of the jump; an early kicker bridges the gap until `harnessd` runs. A slot handed control twice without confirming is skipped.

Power-on to SSH: 3 to 4 minutes. A network push of the image takes about a minute.

Figure 9.2. The boot chain. Stage-0 never writes the user microSD: the image is either read from a slot or pushed over the network into memory.

- **Stage-0** (`src/linux_soc/hw/fw_stage0/`) lives in the processor's 128 KiB local memory, baked into the base bitstream. It waits for the board's clocks to settle, calibrates the DDR4, reads the card's boot-select sector and loads the default OS slot, then the other one.
- **Try once, then confirm.** A slot handed control twice without Linux confirming a healthy boot is skipped; a power cycle clears the count. `harnessd` confirms only when the network is up, SSH is listening and the card path works.
- **Rescue.** With no card, no bootable slot or failed calibration, stage-0 waits. It answers ping, `identify` ("mode": "rescue" with a reason) and TFTP on UDP 69, where an upload is checked and booted once from memory. Nothing answers the control port. The harness serial console prints `stage0: RESCUE (...): <board-ip> -- ping, TFTP put <boot image> (port 69), identify 6899`.
- **The watchdog** resets the processor 21.5 to 43 s after its last kick. Stage-0 arms it before it jumps to Linux, so the kernel and `harnessd` together have 42.9 s to start kicking; an early kicker in the first start-up script bridges the gap for at most 150 s. A hung `harnessd` stops kicking, and the board reboots by itself.

9.3 The user microSD

WHERE	SIZE	HOLDS
Sectors 1–2	2 × 512 B	The boot-select sector, two copies: the default slot and a sequence number
Partition 4 (type <code>0xDA</code>)	32 MiB	The design store: one design kept for power-on (§10.7)
Partition 1	64 MiB	OS slot A: one Linux image
Partition 2	64 MiB	OS slot B: one Linux image
Partition 3 (ext4)	The rest	<code>/persist</code> : the SSH host key, the claim, the identity override and local settings

Stage-0 only reads the card; Linux writes it. A blank card is laid out by Linux running from memory (§1.4), and the harness never formats a card by itself. With no card, or with the kernel argument `mps3.persist=off` , `/persist` lives in memory and is lost at the next reset, taking the claim and the host key with it.

OS slots. A new image is written into the slot that is neither running nor the default, read back through stage-0's own loader, and becomes the default only on `commit` ; `rollback` returns to the other slot. Each pushed file carries a header naming the static shell it was built for, and the board refuses an image for another one. The push is slow (41 to 50 minutes), because the card is written and then read back through the boot loader's path. On a claimed board, push, commit and rollback are accepted only through the board's SSH (§9.5).

9.4 One service, thirteen tasks

`mps3-harnessd` is one process running one loop (`src/linux_harness/sw/harnessd/main_linux.c`) that calls each of its tasks in turn; each does a bounded amount of work and returns. Three overruns of its budget in a row mark a task sick: it is skipped and re-probed every 100 ms, so one misbehaving task cannot starve the control port.

#	TASK	BUDGET	JOB
0	ident	5 ms	Answer the identify probe on UDP 6899
1	kmsg	5 ms	Copy kernel messages into the harness log
2	persist	none	Save the loaded design's clearing file (§10.3); reap the OS-slot verifier
3	swap	50 ms	One step of the load state machine (§10.3)
4	cfgagent	50 ms	Bitstream receivers: TFTP (UDP 69) and raw TCP (6910)
5	ctrl	10 ms	The JSON control channel (6900)

#	TASK	BUDGET	JOB
6	jtag	10 ms	The JTAG server (6921, chapter 11)
7	xvc	50 ms	The XVC server (2542)
8	uart	10 ms	The design's consoles and SWO (6930–6932)
9	clcd	30 ms	The panel, touch, the panel hand-over and Identify
10	diag	2 ms	Refresh the diagnostic mailbox (§9.8)
11	hbeat	1 ms	Heartbeat LED
12	usd	none	The design store on the card (§10.7)

If `harnessd` exits, `init` restarts it at once; if it hangs, the watchdog reboots the board. The `reboot` command restarts the harness through the watchdog, and is refused while a card job runs. **After any restart the partition stays in reset until the next load**, so one `greybox` load is the first step after any harness restart.

9.5 SSH, the claim and the claim lock

`dropbear` serves SSH on port 22 with keys only; the root password works only on the harness serial console. Which keys it trusts is decided by the **claim**, trust on first use: a freshly booted board trusts no key, and the first TFTP write of a file named exactly `authorized_keys` claims it. Every later claim is refused; only `mps3-unclaim` at the serial console releases one. A board in rescue cannot be claimed: stage-0 would take the upload as a boot image and reject it. The claim and the host key (generated at first boot, never shipped) live in `/persist`.

The claim lock. On a claimed board, JTAG (6921) and XVC (2542) answer only the board's own `127.0.0.1`, and the commands that change the card or the board's identity (store commit and the `usd` actions, slot push, commit and rollback, `identity_set`) are accepted only from the board itself. Tools reach them through an SSH port-forward, which Harness Manager opens for you. A connection from elsewhere gets one `... locked: board claimed (use ssh)` line. Loading a design, reading the board and restoring `greybox` stay open to anyone who can reach it.

9.6 The board's identity

One image serves every board, so each board's label, host name, static address and MAC are resolved at every boot, field by field, from the first of: the override `/persist/etc/mps3/identity` (read only when `/persist` is on a card; written by `board identity` or `mps3-identity set` over SSH); the values baked into the board's stage-0; the defaults. Chapter 3 is the site team's side.

A board that runs without a card has no override, so its identity must be baked into its stage-0. A file `boards/<name>.mk` beside the stage-0 source sets `S0_LABEL` (at most 8 characters in a bake), `S0_IP` and `S0_MAC`; stage-0 is built with `S0_BOARD=<name>`, which validates the values, and baked into the release's base bitstream with `updatemem`, exactly as the release's own stage-0 is (§12.4). The result is a per-board base for that board's config SD.

9.7 Ports and protocols

PORT	SERVICE	ON A CLAIMED BOARD
6900/tcp	Control: one JSON request and reply per line	Open; one client at a time
69/udp, 6910/tcp	Bitstream and OS-image pushes (TFTP; raw TCP); the claim (TFTP)	Loads open; slot and store pushes through SSH
6899/udp	Identify: mode, address, claim, host key; answered by stage-0 too	Open

PORT	SERVICE	ON A CLAIMED BOARD
6921/tcp	JTAG, OpenOCD <code>remote_bitbang</code> (chapter 11)	The board itself only
2542/tcp	XVC into the design's debug hub	The board itself only
6930, 6932/tcp	The design's console (UART0) and SWO trace, raw bytes	Open; one client each
22/tcp	SSH, key-only	The claim's keys
3333, 4444, 6666	OpenOCD's GDB, telnet and Tcl ports, on the board's <code>127.0.0.1</code>	Through SSH

The wire protocol is `docs/contracts/net-protocol.md`, version 0.17. The `version` reply carries `"impl": "linux"` and a list of features (`usd`, `slot`, `xvc_lock`, `identity`, `locate`, `panel`, ...); tools test features by name, never by version number. Each port takes one client; an extra connection is accepted and closed at once, which a tool should treat as "busy", not "down". The design's console needs host bytes paced at about 20 ms per character, because the design's receiver has no FIFO.

9.8 Mailboxes and logs

- **The stage-0 status block** at `0x0001_FE00` in local memory records the fabric's `static_id` and `USR_ACCESS` as baked into stage-0, the slot booted and its attempt counters, the calibration result, the rescue reason and the baked identity.
- **The diagnostic mailbox** at `0x0001_FF00` holds the harness's counters for the network, loads, the ICAP and the tasks.
- **The harness log** is a 4 KiB ring of the service's console output, with kernel messages copied in.

Both blocks survive a watchdog reset. Read them with `pyverify mailbox --what stage0|diag|both` (over SSH) or `pyverify netboot --status` in rescue; read the log with `pyverify log --host $B`, and the kernel's with `dmesg` over SSH. Never read them by halting the processor over JTAG (§5.2).

10 The partition

A load replaces the design in the partition while the harness keeps running. This chapter describes the wires a design meets, what a load ships and the order it happens in, how the partition is fenced off meanwhile, the identities that make it safe, how the boundary is timed, and the design kept on the card for power-on.

10.1 The boundary and the partition

The boundary is where the shell ends and a design begins. Every design links into the same black box, so a wrapper that differs by one bit does not degrade gracefully: it fails, often after a full place and route.

- **One source.** `fpga/shell/boundary.yaml` defines it. `tools/gen_boundary.py` generates from it the partition stub, the wrapper skeleton new designs start from, the checker `fpga/dfx/pin_check.py`, and the tables in `docs/contracts/partition-pins.md`. Directions are stated from the shell's side.
- **Its shape.** 47 ports and 148 bits in eight groups: clocks and resets, JTAG, the logic-analyser BSCAN group, RMII and MDIO, the console streams, status (with the 32-bit `rm_id`), 16 GPIO bits, and QSPI. *User Guide* ref E lists every signal.
- **Why it is narrow.** No bus crosses, so a wedged design cannot hang a bus the shell needs; only slow scalars and byte streams cross, and the shell synchronises every crossing on its side. The one exception is QSPI, whose read data is captured against the flash clock: it passes straight through to the pads as a matched group. Clock generation and pad registers belong to the static, a DFX rule. A BSCAN primitive is illegal in a partition, so a design with logic analysers includes a debug bridge fed by the BSCAN group.
- **The partition** is the pblock `pblock_rp_dut` over clock regions X2Y0, X3Y0, X2Y1 and X3Y1 in SLR0, with no I/O sites. It is never empty: with nothing else loaded it holds `greybox`, which ties every output off and drives `rm_id` 0.

`make -C fpga/dfx pin-check` checks every registered wrapper against `boundary.yaml` in about a second, with no Vivado. Outside the repository, the build kit carries the boundary, and Harness Manager checks a wrapper against it (*User Guide* ch 7).

10.2 The overlay

Every design ships as three files in `overlay/<name>/`, written by `fpga/dfx/gen_manifest.py`: the **partial** (the design's fabric); the **clearing** file, which returns the partition to blank before the *next* design goes in; and **manifest.json**, which names the `static_id` it fits, the `rm_id` and name, each payload's length and CRC-32, and the shell's USERCODE. A design with logic analysers also carries its `.ltx` probe file, paired to the partial by CRC, because a probe file is valid only for the routed build it came from. The schema is `docs/contracts/overlay-manifest.md` (v0.4).

10.3 A load, step by step

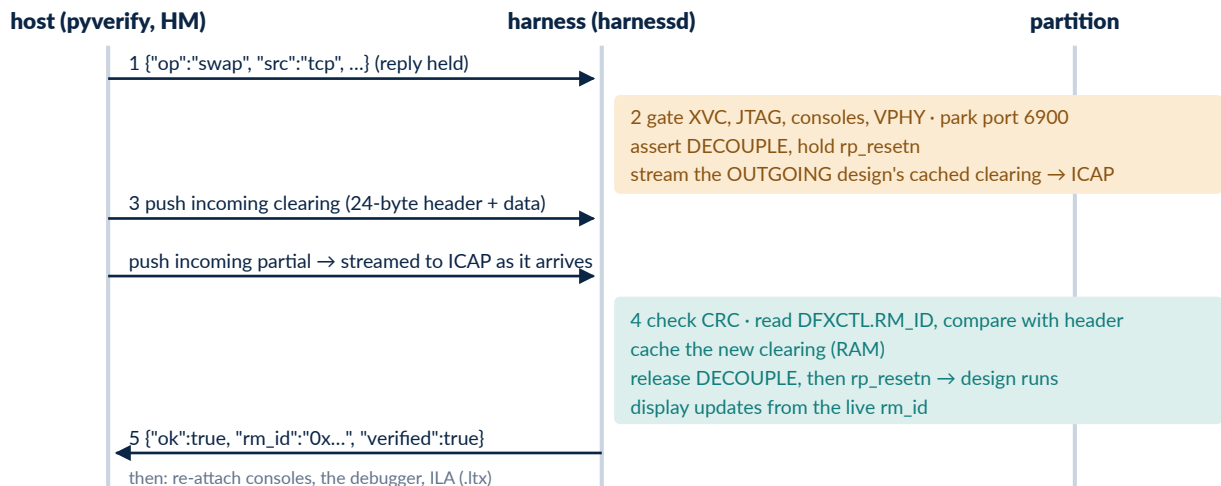


Figure 10.1. The order of a load, as the protocol requires it (`docs/contracts/net-protocol.md`, "Swap sequence"): the swap request first, then the push, then the held reply. A push that arrives outside a load is refused.

- **The harness owns the clearing sequence.** It keeps the clearing file of whatever is loaded now, in a 1 MiB buffer in memory, so the host sends only the incoming pair. At power-on the resident clearing is `greybox`'s, which ships in the Linux image. The incoming clearing replaces the resident one only after the new design verifies, so a failed load never loses the still-valid one, and it is saved to `/persist` as well, so a restarted `harnessd` can still clear the running design.
- **The header.** Each pushed file carries 24 bytes: `"MPS3"` | u16 version | u8 kind (0 clearing, 1 partial, 2 OS image) | u8 slot | u32 static_id | u32 rm_id | u32 length in words | u32 CRC-32. A header for another `static_id` is rejected before one word reaches the ICAP.
- **The state machine** is stepped once per pass of the service loop, so the control port keeps answering throughout. A whole load, push included, takes 35 to 86 s. Afterwards every connection to the design is stale: the consoles are flushed, the debug port and logic analysers are new, and the board's OpenOCD has been stopped (§11.3).

10.4 Fencing the partition

A DFX decoupler with 20 interfaces clamps every partition-to-shell signal while a load runs. Every clamp value is 0 except the flash's chip select, which is held at 1 so that the flash stays deselected. Shell-to-partition signals are not clamped: the partition is held in reset for the whole load instead. `DECOUPLE` is cleared only by a board power-on, and a watchdog reset sets it, so a supervisor that dies leaves the partition isolated rather than loose.

10.5 Identity

IDENTITY	ANSWERS	HOW IT IS MADE AND CHECKED
<code>static_id</code>	Will this partial fit?	The CRC-32 of the locked static checkpoint <code>static_routed_locked.dcp</code> . It changes on <i>every</i> rebuild of the shell, even from unchanged sources, because the checkpoint embeds timestamps. Every overlay, pushed header and build kit is keyed by it.
<code>rm_id</code>	What is running?	A constant the design drives on its <code>rm_id</code> pins: <code>{major[31:24], minor[23:16], design_id[15:0]}</code> , derived by <code>rm_id_of</code> in <code>fpga/dfx/rm_list.tcl</code> , never typed. The boundary clamps it to 0 during a load, so reading the expected value back afterwards is a real confirmation. <code>greybox</code> is 0.

IDENTITY	ANSWERS	HOW IT IS MADE AND CHECKED
USERCODE	Which implementation run?	The full bitstream's <code>USERID</code> , carrying the build commit; stamped into every manifest as <code>static_usercode</code> . Read over JTAG or from the <code>.bit</code> header.
USR_ACCESS	Which harness version?	The repository's <code>VERSION</code> file packed as <code>{major, minor, patch, flags}</code> , stamped into the bitstream and the Linux image. It changes only with a new static, so it can lag the release tag (§2.3).
static_canon	Did the static really change?	A SHA-256 over a canonical list of the inputs that decide the static (part, Vivado release, build flags, the sha256 of every declared source), with no timestamps. <code>make -C fpga/dfx canon-check</code> fails if a design source is undeclared.

The identity lock. The Linux image lives on a card and can change independently of the bitstream, so the `static_id` a board reports is the one baked into stage-0 with the shell, published in its status block. If that block is missing, if the image was built for another static, or if the fabric's `USR_ACCESS` disagrees with the image's release, the harness **locks**: `version` carries `id_skew` with the reason, and loads and store commits are refused with `identity lock: <reason>` before anything is touched.

10.6 Timing at the boundary

The shell drives two clocks across the boundary: `dut_clk` (50 MHz: the harness can retune it, but the designs are timed for 50 MHz and Harness Manager refuses a change, §7.5) and the fixed 50 MHz RMII reference; the logic-analyser group adds a 12.5 MHz BSCAN clock. `docs/contracts/partition-timing.md` fixes their periods, and each design constrains itself out of context to match.

Boundary timing (known issue 11, closed for the v2.0.0 designs). When a design's out-of-context constraints *create* `dut_clk` in its own checkpoint, the shell's clock definitions drop out at link, and the static-to-partition paths go unanalysed. The timing report then shows a large `checking no_clock` count, while its summary still says that every constraint is met. The locked static is fully timed and every design is timed internally; only the boundary paths are missed. Eight of the thirteen released designs were built this way (the nanoSoC family, `eth_ss`, `socscope`, `clcd_demo`, `dbg_demo`). For v2.0 they were re-timed with the shell's clocks restored, without changing the bitstreams, and every same-clock boundary path meets setup and hold (worst hold +0.212 ns), so the published designs are timed. Two pre-existing items remain outside the boundary: the Ethernet management clock model in `eth_ss` and `nanosoc_multicore`, and a harmless static-internal path in the designs with logic-analyser ports. Kit builds made with Harness Manager 1.0.0's build script are not affected: the script writes the design's checkpoint before it reads the out-of-context constraints, so the boundary is timed (a full build went from 27,984 unlocked pins to 0).

Harness Manager checks for it, for builds made with an older script or from a synthesis checkpoint you bring. `kit check` and `kit build` (and the Build tab's Check step) read Vivado's `check_timing` section and warn when any register pin has no clock or more than 1,000 endpoints are unconstrained: `boundary not timed (known issue 11): ... so the static<->RM partition boundary was not timing-analysed, even where the summary says every constraint is met. Your RM's own paths were still timed. The fix is to rebuild with a build_rm.tcl written by the current Harness Manager (harness-manager kit script ... --out DIR). For your own builds in the repository, never create_clock dut_clk in a design's XDC, and check no_clock is 0 before you trust a timing report.`

10.7 Keeping a design on the card

The harness can keep one design on the user microSD and load it by itself at power-on. The store is partition 4 (type `0xDA`, found by type, not position). Inside are a ping-pong header (two copies, each with a sequence

number and CRC; readers take the newest valid copy, writers overwrite the other) and two 8 MiB slots, each holding one design's clearing and partial. A card with no such partition is reported `foreign` and never written.

- **Committing** keeps only what is running: its `static_id` and `rm_id` must match the live ones. The pair is pushed again into the inactive slot, read back and checked, and only then does the header flip. A failure at any point leaves the previous default in place.
- **Loading at power-on** happens once per FPGA configuration, after the network is up, through the ordinary load state machine, and only if the store is valid, the slot's `static_id` is the shell's, both CRCs pass before a byte reaches the ICAP, and `USER_nPB1` is not held. A harness restart never reloads it.

KNOWN ISSUE 12: THE POWER-ON LOAD DOES NOT COMPLETE IN V2.0.0

After a power-on or an MCC reboot, the harness must read and check the whole kept pair before it may load it. Under Linux the card reads at about 14 KB/s, so a `nanosoc` pair takes about 130 s, longer than the 30 s the power-on load allows. `card status` then shows `power-on failed:timeout` and the board stays on greybox. The kept pair stays on the card. After a power cycle, load the design again. A later release moves card reads off the harness loop and fixes this.

The tools are Harness Manager's `program ... --keep-on-card` and `card status|commit|clear`, and `pyverify` `usd`. The wire is `net-protocol.md`, "User microSD"; the format is `overlay-manifest.md`, "A/B slot store".

11 The on-board debug path

A user debugs a design's processor with only Arm GDB on their PC: OpenOCD, the JTAG adapter and the JTAG wires are all on the board. This chapter follows a GDB request from the PC to the design's Cortex-M core and back, describes the launcher that runs OpenOCD on the board and the contract it keeps, and covers the logic-analyser path over XVC.

11.1 The chain

#	HOP	WHAT HAPPENS THERE
1	GDB on the PC	Connects to a local port that Harness Manager forwards over the claim's SSH connection.
2	OpenOCD on the board	Serves GDB on the board's <code>127.0.0.1:3333</code> (3334 for a second core). It holds the JTAG state machine and the CoreSight and Cortex-M logic, and uses the <code>remote_bitbang</code> adapter.
3	harnessd's JTAG server	Listens on 6921 (the board's own address only, once claimed). Each byte from OpenOCD sets TCK, TMS and TDI (the characters <code>'0' - '7'</code>) or asks for TDO (<code>'R'</code>); the server turns each into a register write or read.
4	JTAGBB	The shell's bit-bang block at <code>0x44A7_0000</code> : <code>DRIVE</code> sets the pins, <code>SAMPLE</code> reads TDO.
5	The boundary	The <code>jtag_*</code> partition pins. TCK here is a firmware-paced signal, not a clock in the static.
6	The design	The nanoSoC's CoreSight SWJ-DP (IDCODE <code>0x6ba00477</code>), its AHB-AP, and the Cortex-M0.

The shell is the JTAG adapter; nothing is brought out to a connector. The JTAG server takes one client at a time, and that rule is the whole of the exclusivity: whoever holds 6921 holds the design's debug port.

11.2 OpenOCD on the board: `mps3-debug`

The Linux image carries OpenOCD 0.12.0, built with the `remote_bitbang` adapter and the platform's `hostio4` and `ahb_qspi` drivers, and a launcher, `/usr/bin/mps3-debug`, run as root over the claim's SSH. Its interface is a contract, `mps3-debug/1`, so that tools can rely on it:

- **Verbs:** `up` [`--rm` `auto` `NAME`], `down`, `status` and `version`, each printing one JSON object: the state, the design and its configuration, the process, the TAP's IDCODE, each core's name and GDB port, the telnet and Tcl ports, who is busy on 6921, any `error` (code, message, hint) and the tail of OpenOCD's log.
- **Exit codes:** 0 ok; 4 busy (another client holds 6921, and `busy` names it); 6 failed (OpenOCD exited, with its last lines); 12 no OpenOCD in the image; 13 the loaded design has no debug port; 14 no configuration for the design.
- **The process** runs as the unprivileged `openocd` user, against `127.0.0.1:6921`, and serves GDB on 3333 (and 3334), telnet on 4444 and Tcl on 6666, all on `127.0.0.1`.
- **Choosing the configuration:** `--rm auto` reads the loaded `rm_id` from the harness's own identify reply and maps it through `/usr/share/mps3/openocd/designs.conf`; Harness Manager passes the design's name instead.
- **The session ends** on `down`, when OpenOCD exits, after 2 hours with no GDB client, and when the `rm_id` changes, because a load rebuilds the debug port. The state and OpenOCD's log are in `/run/mps3-debug/`.

11.3 How Harness Manager drives it

`harness-manager debug up $B` runs the launcher over SSH, forwards one random local port per core (only the GDB ports), and prints the line to attach with. It holds the session until Ctrl-C:


```

debug      mps3@192.168.10.101:6900: up
where      the board: OpenOCD runs there; gdb reaches it through the board's SSH
gdb        127.0.0.1:43571
attach     arm-none-eabi-gdb -ex "set remotetimeout 60" -ex "target extended-remote
127.0.0.1:43571"
config     nanosoc_mps3_jtag.cfg nanosoc_ops.tcl

```

`debug status`, `debug down` and `debug detect` (the TAP's IDCODE, without starting a session) complete the set. Before every `program` or `restore` on a claimed board, Harness Manager stops OpenOCD on the board (`mps3-debug down`), so that it cannot drive JTAG while the partition is rewritten. If that fails, the load is refused (exit 15) and nothing is programmed; `--force` loads anyway, with a warning. A terminal holding `debug up` through a load says the session closed for the swap and, once the load verifies, prints the new ports to attach to.

11.4 Speed, and what comes next

Every JTAG bit crosses a TCP socket inside the board and a register access, so memory access over this path is slow in v2.0.0 (about 20 bytes a second): halting, stepping, breakpoints and reading registers are fine, but loading a program into the design's memory takes minutes. **v2.0.1 and later** carry a faster OpenOCD in the Linux image (§2.5). A later release plans an adapter that drives JTAGBB directly from OpenOCD, without the socket.

11.5 Logic analysers over XVC

Vivado's `hw_server` speaks XVC to the harness on TCP 2542; the harness drives the debug bridge at `0x44A8_0000`, whose BSCAN outputs leave the shell as the 12-wire `dbg_bscan` group; inside the design, a debug bridge and hub fan out to its ILAs. That hub is clocked from the fixed 50 MHz RMII reference, never the DUT clock, so it keeps working when the DUT clock changes. Harness Manager's `xvc open` takes the board's one XVC slot, runs its own `hw_server` behind a local relay (Vivado 2026.1 for this release's designs), reopens the session after every load, and hands Vivado the design's `.ltx`.

XVC reaches the partition's debug chain only, never the whole device's JTAG. A static-side ILA is forbidden, because it would tie the static to one design; the one piece of static debug logic is the DDR4 controller's calibration-debug core, reached through the bridge's second BSCAN output, so Vivado shows the memory's calibration view beside the design's own hub. That view exists only while Linux runs.

11.6 An OpenOCD on the PC

An OpenOCD on a PC can drive 6921 instead, if it is a build that includes `remote_bitbang` (xPack OpenOCD 0.12.0-7 does). On a claimed board it reaches 6921 through an SSH forward to the board's `127.0.0.1`. This is the route for the bare-metal shell, which has no OpenOCD of its own (Appendix C), and a fallback for a Linux image without the launcher. Close the forward after a failed run, or it keeps holding 6921, and leave 2 s between sessions.

11.7 Rules

- **Never halt the harness's own processor** through a JTAG cable on J17: Linux stops, and the watchdog resets the board about 43 s later.
- **`nanosoc_multicore`**: the board's configuration debugs core 1. Core 0 and its debug logic stay in reset until core 1's boot ROM opens its boot gate, so OpenOCD reports `Cortex-M PARTNO 0x0` is unrecognized for core 0; two-core debugging is not proven.
- **Busy is not down.** `mps3-debug up` exit 4 names the client holding 6921 (a PC OpenOCD, or a forward left open): stop it and retry. Exit 6: read `log_tail`, then `/run/mps3-debug/openocd.log` on the board.

12 Building and releasing the harness

Four things are built for a release: the static shell with stage-0 baked in, the Linux image, the designs, and the overlays packaged from them. Rebuilding the static shell takes hours and strands every design built for the previous one, so most work avoids it (chapter 13). A design author outside the repository needs only the build kit (§12.6, and *User Guide* ch 7).

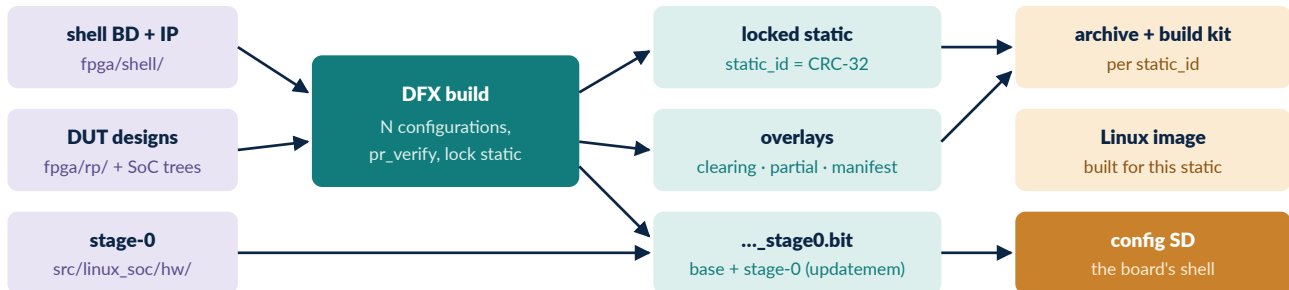


Figure 12.1. What building a new static shell produces. The Linux image is built afterwards, for that static, and reaches a board through the user microSD or the network, never the config SD. A new design can be added later against the same static without rebuilding it (§13.1).

12.1 The environment

```
source set_env.sh           # Python, cocotb, simulators (site-specific parts in
set_env.local.sh)
cp tools.env.example tools.env # untracked: paths to the tools and the SoC source trees
```

`tools.env` is needed only to build designs and shells, not for the checks. Its variables have no defaults, on purpose: `S0CLABS_NANOSOC_SOC_DIR` and `S0CLABS_AHB_QSPI_DIR` (the single-core nanoSoC tree and its QSPI controller), the other designs' trees, `ARM_IP_LIBRARY_PATH` (your Arm IP, read in place and never copied), `VIVADO / VIVADO_VER`, and `MINT_HUB` (where a new static's outputs are archived, one directory per `static_id`).

One build directory, one Vivado. A checkpoint opens only in the Vivado release that wrote it, so the Makefile checks the release before any Vivado stage and refuses to mix them. This release's shell needs Vivado 2026.1, for its MicroBlaze V IP. The Linux image is built with Buildroot by `src/linux_harness/sw/build.sh`, in its own tree.

12.2 The SoC generator

A nanoSoC is described once, in YAML (cores, memories, bus targets and addresses, peripherals, debug, boot), against the architecture library `nanosoc_arch_tech`, a submodule of each SoC project. Its generator, `nanosoc_gen`, validates the description and writes into `build_soc/` the SoC top, the bus matrices, register blocks, memory-map headers, linker fragments, an ASIC pad-ring top and documentation. `build_soc/` is regenerated, so a change goes into the YAML or a template, never into the output. Between simulation, the FPGA and silicon only three seams change: memory macro wrappers, the pad ring and clock generation. The platform's only FPGA-specific RTL for a design is its partition wrapper.

12.3 The build targets

MAKE -C FPGA/DFX ...	PURPOSE
<code>pin-check</code>	Every wrapper against the boundary, about 1 s, no Vivado
<code>rm-<name>-dcp</code>	Synthesise one design out of context

MAKE -C FPGA/DFX ...	PURPOSE
<code>add -rm-<name></code>	Implement one design against the locked static; keeps the <code>static_id</code>
<code>overlays, verify</code>	Package the overlays; check every manifest's CRCs
<code>prod</code>	The full multi-configuration build: a new static
<code>mint</code> (or one stage, <code>mint-preflight ... mint-hub-copy</code>)	A whole new static shell, with everything that depends on it (§12.4)
<code>mint-stage0</code>	Build stage-0 for this static and bake it into the base bitstream
<code>linux-image-env, mint-linux-image, linux-bundle-check</code>	The static's identity for the image build; pack the image into the release bundle; check the bundle (§12.5)
<code>fielded-files</code>	The set of files that preserves a shell: the locked static, stage-0, the bundle and the image's licence archive
<code>canon-check, overlay-usercode-check</code>	<code>static_canon</code> coverage; USERCODE agreement between the static and the manifests (§10.5)

12.4 Building a new static shell

A new static shell is built in one run of eight stages; preview it with `make -n` first:

```
make -n -C fpga/dfx mint SHELL_CPU=mbv BUILD=$PWD/fpga/dfx/build_<name>
SHELL_PROJ=$PWD/fpga/dfx/build_<name>/shell_proj \
  RM_SET="greybox regdemo_a regdemo_b led uart_echo nanosoc eth_ss nanosoc_multicore socscope
nanosoc_upy clcd_demo dbg_demo nanosoc_ila" \
  SHELL_TOUCH=1 TOUCH=1 DFX_JOBS=4
```

#	STAGE	PRODUCES
1	preflight	Refuses inconsistent settings, such as <code>SHELL_TOUCH=1 TOUCH=0</code> or the wrong Vivado
2	shell	The static shell's synthesised checkpoint and its hardware hand-off (<code>.xsa</code>)
3	designs	Every design's out-of-context checkpoint
4	prod	All configurations placed and routed, <code>pr_verify</code> , the locked static and its <code>static_id</code>
5	overlays	The overlays, keyed to the new static
6	stage-0	Stage-0 built for this <code>static_id</code> and <code>USR_ACCESS</code> , baked with <code>updatemem</code> into <code>config_rm_greybox_stage0.bit</code> , with <code>stage0_bake.json</code> ; a stage-0 built for another static is refused
7	record	<code>mint.json</code> : every input and result; a missing value is null with a reason
8	archive	The outputs copied to <code>\$MINT_HUB/<static_id>/</code>

Preserve the locked static first. `static_routed_locked.dcp` is the only input that can ever add a design to that shell, and the build kit is made from it. Keep each one in a directory of its own, named by its `static_id`, before the next build can overwrite it. A stage-0 change alone is baked into an existing base with `updatemem`: only the local memory's contents change, so the `static_id`, `USERCODE` and `USR_ACCESS` stay the same and every design still loads.

12.5 The Linux image and the release bundle

```
make -C fpga/dfx mint SHELL_CPU=mbv BUILD=<b> SHELL_PROJ=<b>/shell_proj SHELL_TOUCH=1 ...
eval "$(make -s -C fpga/dfx linux-image-env SHELL_CPU=mbv BUILD=<b>)" # the static's identity,
for the image
src/linux_harness/sw/build.sh # OpenSBI, Linux, harnessd
make -C fpga/dfx mint-linux-image SHELL_CPU=mbv BUILD=<b> SHELL_PROJ=<b>/shell_proj
```

- **The image** (`linux_slot.img`) is built for one static, from the built tree, which also hands it `greybox`'s clearing file. `mint-linux-image` refuses an image built for another static, and so does the board. A new image changes neither the static nor the designs.
- **The release bundle** (`linux_bundle.json`) names every byte of a release and the route it takes: the base through the config SD, the image and the overlays over Ethernet. It records the static's four identities (`static_id`, `USERCODE`, `USR_ACCESS`, `static_canon`) and travels with `linux_legal_info.tar`, Buildroot's licence information.
- **The config-SD folder** is assembled by `fpga/mps3_sd/assemble_sd.sh` ALL `<base.bit>` from `fpga/mps3_sd/templates/: config.txt` at the root and one `MB/HBI0309<rev>/` tree per revision, which differ only in their `BOARD:` line. Ship `UARTMODE: 0`, as the release does.

12.6 The build kit

A design author outside the repository builds against the **build kit**: one zip per static shell, made from its archive and attached to that release on GitHub (<https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>). Harness Manager imports the downloaded zip (`kit import`), checks it, writes the Vivado script, and checks the result (*User Guide* ch 7). The build uses three of its files:

- `static/static_routed_locked.dcp`: the routed static with the partition empty and the routing locked. It already holds the floorplan and the shell's constraints, so a user never floorplans. Its CRC-32 is the `static_id`.
- `boundary/`: the 47 ports and 148 bits.
- `kit.json`: the part, the partition cell and pblock, the exact Vivado release, and every file's sha256.

The kit carries the full locked static because AMD's lighter *abstract shell* does not support UltraScale: Vivado refuses it on the KU115. The XDC a design builds with is an out-of-context file scoped to the design, not the shell's XDC, whose constraints are already in the locked checkpoint. What Vivado runs is UG909's non-project flow for a further configuration against a locked static:

```
synth_design -mode out_of_context -top <your top> -part xcku115-flvb1760-1-c
read_xdc <name>.ooc.xdc
write_checkpoint <name>_synth.dcp
open_checkpoint static_routed_locked.dcp
read_checkpoint -cell u_rp_dut <name>_synth.dcp
opt_design; place_design; phys_opt_design; route_design
write_checkpoint <name>_routed.dcp
pr_verify static_routed_locked.dcp <name>_routed.dcp
write_bitstream -bin_file -cell u_rp_dut <name>_partial
```

The generated script wraps this in about 25 kinds of gate, each printing `HM_GATE` lines (one per source file for the source check, so a large design prints hundreds): the inputs and the Vivado release, the checkpoint's CRC against the `static_id`, no black boxes, the 148 boundary bits, the `rm_id` the netlist drives, the clocks, the partition pins after link and after optimisation, the DFX design-rule checks, the design's timing, `pr_verify`, the outputs, and a clearing file that fits the harness's buffer. The verdict is the receipt's `state`, never Vivado's exit

code, which is 0 even when a gate fails. The flow reproduces the platform's own overlays: `nanosoc` built this way is byte-identical to the release's.

12.7 Verification before a release

- **The repository gate.** `make check-ci` runs the board-free stages in about 3 minutes: the contracts present, the agreement checks (boundary, `rm_id`, manifests, generated-file freshness), the host tests, and the firmware and harness logic on the host compiler, including `make check-linux`, which holds the bare-metal firmware and `harnessd` to the same bytes on the wire. `make lint` runs Verilator on the shell RTL. `make check SIM=vcs` runs everything; a run that skipped a stage ends `CHECK OK -- but these stages did NOT run: ...`. Run the gate locally before any merge, whether or not hosted CI ran.
- **Testing without a board.** `FakeShell` (`host/pyverify/pyverify/testing/fakeshell.py`) is a byte-conformant model of the harness's network surface, held to the harness's own encoder by a conformance test. A new command lands in the harness, `pyverify` and `FakeShell` together.
- **On a board.** `scripts/harness_regression.sh` is the go/no-go for a newly built harness: static checks, implementation reports (positive slack, no DRC errors), then, with `--allow-board`, register liveness, the mailbox, loads, a console round trip and a nanoSoC halt over JTAG. Its register check never halts the harness's processor.
- **The release gate** is a 24-hour soak PASS (§4.3), with every design loaded in turn.

A check that cannot fail is not a check: each gate names the fault it would catch and is shown to fail on a seeded one. A verdict on a board comes from the board (a console line, a verified load with the exact `rm_id`), never from a tool's own "ok".

13 Changing the harness safely

Some changes reach a board as a new Linux image in an afternoon; others strand every design anyone has built. This chapter sorts changes by what they cost, gives the procedure for the expensive ones, and lists the traps that have caught people changing the harness before.

13.1 What each kind of change costs

YOU CHANGE	NEW STATIC?	WHAT MUST FOLLOW
<code>harnessd</code> , a Linux service or script, the kernel configuration, OpenOCD on the board	No	A new Linux image for the same static, pushed into an OS slot (§2.4). Every design keeps working.
Stage-0, or a board's baked identity	No	Bake the new stage-0 into the existing base with <code>updatemem (mint-stage0)</code> , then a config-SD write and an MCC reboot on every board. The <code>static_id</code> , <code>USERCODE</code> and <code>USR_ACCESS</code> stay the same.
A new design that fits the boundary	No	Implement it against the locked static (<code>add -rm -<name></code> , or the build kit). <code>pr_verify</code> proves the static is unchanged.
The network protocol (a new command or reply field)	No	The contract, <code>harnessd</code> , <code>pyverify</code> , <code>FakeShell</code> and Harness Manager change together (§13.4); a new image.
Any shell RTL, the block design, a constraint, an IP version, the Vivado release, or a rebuild of the shell from unchanged sources	Yes	A new <code>static_id</code> . Every overlay is rebuilt, every user's design is rebuilt with the new build kit, a Linux image is built for the new static, and every board's config SD is rewritten (§2.6).
The partition boundary	Yes	All of the above, and every design's wrapper changes, including your users' (§13.2).
A new shell block	Yes	A new 64 KiB window in the register map, and all of the above (§13.3).

The rule behind the table: **any change to the static shell produces a new `static_id`, and every design must then be re-keyed.** The checkpoint embeds timestamps, so even an identical rebuild changes the id. `static_canon` tells you whether the inputs really changed (§10.5); the `static_id` never forgives a rebuild.

WHY

A partial bitstream is placed and routed against one exact static. Loaded on any other, it would drive wires that are not there. The `static_id` check is what turns that silent corruption into a refusal, so it is never bypassed.

Batch static changes. Decide in advance what goes into the next static, build it once, and release it with its overlays, its Linux image, its build kit (attached to the GitHub release) and release notes that tell users to rebuild. Keep the previous static's locked checkpoint and kit archived: boards and users may need it until they move.

13.2 Changing the boundary

- 1 **Edit** `fpga/shell/boundary.yaml`, and nothing else by hand. Directions are from the shell's side.
- 2 **Regenerate** with `tools/gen_boundary.py`: the partition stub, the wrapper skeleton, `pin_check.py` and the tables in `docs/contracts/partition-pins.md`. The freshness check fails if any of them drifts.
- 3 **Update every wrapper** in `fpga/rp/`, and `greybox` first: it must tie every new output off. `make -C fpga/dfx pin-check` must pass.

- 4 **Update the timing contract** (`partition-timing.md`) if a clock crosses, and the decoupler if a new partition output needs a safe value.
- 5 **Build a new static** (§12.4) with the other static changes you have batched, and release it as §13.1 describes.

Keep the rules of §10.1: no bus, no clock generation, no pads in a design, and every crossing synchronised on the shell's side. A design that needs more than the boundary offers usually needs a better shell block, not a wider boundary.

13.3 Changing the register map

The register map is generated from the block design by `tools/gen_regmap.py` into the contract, the C header and the Python driver; a freshness check fails if any view drifts, and a conformance test holds the hand-written bit-field tables to the generated offsets. A new block also needs an owner under Linux, declared in `MBV_OWNERS` (the kernel or `harnessd`): the generator refuses a block without one. The `0x44A0 – 0x44B4` region is full (its three free pages, `0x44AF – 0x44B1`, are reserved for designed blocks), so a new block means a new window. Bench every block at the parameters the block design gives it, not its RTL defaults: a slave benched with a 12-bit address and instantiated with 32 sees full system addresses, and works on the bench while it is dead on the board.

13.4 Changing the protocol

- **Write the contract first** (`docs/contracts/net-protocol.md`), with the reply's keys in order and every refusal's text.
- **Land it everywhere at once:** `harnessd` (and the shared service modules), `pyverify`, `FakeShell`, and a conformance test that holds the model to the harness's own encoder. A fake more permissive than the harness once kept a test suite green against the wrong load order.
- **Advertise it as a feature** in the `version` reply, and have tools test the feature by name. Never make a tool depend on a version number: `identify`'s `proto` field has lagged the protocol before.
- **Keep one client per port**, and keep every task's work bounded, so that the control port answers during a load.

13.5 Traps when building and changing

SYMPTOM	CAUSE	FIX
A timing report shows a large <code>checking no_clock</code> count, yet "all constraints are met"	The design's XDC creates <code>dut_clk</code> in its checkpoint, so the shell's clocks drop out at link (known issue 11, §10.6)	Take the clock from the shell; check <code>no_clock</code> is 0 before you trust a report
Edits to the generated SoC vanish	<code>build_soc/</code> is regenerated	Change the YAML or the template
The same source builds different firmware	A flags variable on the <code>make</code> command line (such as <code>GNU_CC_EXTRA_FLAGS</code>) replaces every sub-make's own flags	Use the build's appending variable; never pass flags variables on the command line
A "dry run" changed tracked files	An older script's dry-run switch still ran real steps	Preview a build with <code>make -n</code>
A register "bug" in a design or a vendor block	The register layout was assumed, not read; a reference testbench that shares the assumption confirms nothing	Check each field against the RTL before you report a bug
<code>make check</code> says an overlay is STALE	It is keyed to a static that is no longer current	<code>make -C fpga/dfx overlays</code> against the current static

SYMPTOM	CAUSE	FIX
A bake "succeeded", but the bitstream is missing or old	<code>updatemem</code> exits 0 after printing its usage when a path is wrong; Vivado can exit 0 having done nothing	Check the output file and a success line in the log, never the exit code
No design can be added to a shell any more	Its locked checkpoint was lost or overwritten by a later build in the same directory	Preserve each locked static in its own directory, named by its <code>static_id</code> (§12.4)
A Linux image is refused at <code>mint-linux-image</code>	It was built for another static, or it is a development image that bakes an SSH host key	Build it for this static, as a release image
A new image boots, then fails its first load over <code>greybox</code>	The image lacks this static's <code>greybox</code> clearing file, so <code>harnessd</code> fails closed	Build the image from the built tree, which exports the clearing file

14 Harness Manager architecture

Harness Manager is the platform's desktop application: a window and a command line, both clients of one local service, whose engine serves a board on a desk and a board behind a hub alike. This chapter is for people who extend it or debug it; *User Guide* ref A is the command reference, and its repository (<https://github.com/SoC-Labs/HarnessManager>) holds the developer documents.

COMPONENT	WHAT IT IS
Service	<code>harness-manager-daemon</code> , one per user, because a board's session lock belongs to one process and each harness port takes one client. It listens on <code>127.0.0.1</code> only, writes its port and a token to <code>daemon.json</code> (mode 0600), requires the token on every request (HTTP and WebSocket), and runs the overnight checks. Its log is <code>~/ .config/harness-manager/daemon.log</code> .
Command line	<code>harness-manager</code> . It uses a running service, and falls back to an in-process engine. Every verb takes <code>-json</code> or <code>--tsv</code> for scripts, and exits with a fixed set of codes (0 ok, 4 held, 7 unreachable, 12 unavailable, 14 incompatible, 15 refused, and others).
Window	A browser app with five tabs (Overview, Workbench, Build, Board, Checks), built from vendored ES modules with no build step and no network fetch, under a <code>script-src 'self'</code> content-security policy.
Engine	Probing, sessions and locks, the capability view, an event bus, and board-agnostic services: loading, consoles, debug, XVC, leases, XDC export, build kits, the claim, OS slots and the card, the live panel, board identity and the checks.
Board pack	Registered under the Python entry point <code>harness_manager.boards</code> . A pack supplies discovery, the controller driver, the harness protocol, load methods, debug routes, telemetry, pin data and the bundle format. The MPS3 pack is <code>harness_manager_mps3.pack:Mps3Pack</code> .
Transports	Direct (TCP, serial, the config-SD volume, local processes) or a hub (SSH to fpgahub). On a claimed board the MPS3 pack adds an SSH connection to the board for OpenOCD's GDB ports, XVC and the claim-locked commands (§9.5).
pyverify	The one implementation of the harness protocol. It lives in the platform repository (<code>host/pyverify</code>), with <code>FakeShell</code> ; Harness Manager vendors it as a wheel whose sha256 its own tests check. It is not on PyPI: never install a package of that name from there.

14.1 Design rules

- **Standalone first.** It works for someone with one board and no infrastructure; a hub is an optional transport, served by the same engine.
- **USB once, then Ethernet.** The Debug USB is needed to back up the config SD and install the harness; daily use needs only Ethernet.
- **It never guesses.** Every capability is reached through a *route*: a set of links (Ethernet, Debug USB, hub, board SSH, a power plug) plus the features the harness reports. A missing capability always carries its reason, such as `needs the Debug USB cable`; `harness-manager info` lists them.
- **Safety rails are code.** Destructive MCC commands are refused, the config SD is backed up before any write, risky actions need a typed phrase that `--yes` never implies, and nothing reports "installed" until the board confirms it.

It recognises both harness generations (`version.impl == "linux"` marks the Linux harness) and allows a Linux harness 300 s to come back after a reboot, the bare-metal shell 120 s. The temperature it shows from the MCC is the FPGA die's, not the room's.

A Pin reference

The 202 FPGA pins the static shell constrains, taken from the shell's own constraint files:

`fpga/shell/constraints/mps3_harness.xdc`, `optional/mps3_harness_touch.xdc` and `mbv/DDR4_pins.xdc` in the same folder. Those files are the authority: if they and this page ever differ, the XDC is right.

Pins not listed are unused by the shell. A bus is listed from index 0 upwards.

GROUP	PORTS AND PACKAGE PINS	I/O STANDARD	PURPOSE
Clocks	OSCCLK1 AK16	LVC MOS18	Board oscillator into the shell's clock wizards
Buttons	USER_nPB0 AT30 USER_nPB1 AT32	LVC MOS18	PB0 is the shell's power-on reset; PB1 hands the display over
LEDs	USER_nLED[7:0] AU32 AU30 AU31 AR32 AT33 AW30 AW31 AR30	LVC MOS18	Driven by DUT GPIO 7:0 (active low)
DIP switches	USER_SW[7:0] BA29 BB29 BA32 BA33 BA30 BB30 AY33 AY31	LVC MOS18	Read on DUT GPIO 15:8
LAN9220 Ethernet	ETH_INT AK23 ETH_nCS AL24 ETH_nOE AJ23	LVC MOS18	Harness network controller: chip select, output enable, interrupt
LAN9220 bus	SMBF_ADDR[6:0] AK20 AK21 AJ18 AJ19 AH21 AJ21 AH19 SMBF_DATA[15:0] AK22 AL22 AL19 AL20 AH18 AM19 AN19 AP19 AP20 AM20 AN21 AP21 AR22 AM21 AM22 AN22 SMBF_FIF0SEL AJ20 SMBF_nOE AN23 SMBF_nRST AL23 SMBF_nWE AP23	LVC MOS18	The 16-bit static memory bus to the LAN9220
Shell console	MB_UART_RXD AE28 MB_UART_TXD AD28	LVC MOS18	FPGA UART lane 2, host tty_02, 115 200 baud
Touch	CLCD_TINT AJ14 CLCD_TSCL AL18 CLCD_TSDA AJ15	LVC MOS18	STMPE811 I2C and pen-down interrupt (SHELL_TOUCH=1)
Display	CLCD_BL AJ16 CLCD_CS AP15 CLCD_PD[17:10] AN17 AP16 AP18 AR18 AM16 AN16 AR17 AR16 CLCD_RD AM15 CLCD_RS AN14 CLCD_RST AK18 CLCD_WR_SCL AP14	LVC MOS18	HX8347-D 8080 bus, backlight and reset
QSPI flash	QSPI_D0 AU24 QSPI_D1 AV24 QSPI_D2 AV21 QSPI_D3 AV22 QSPI_SCLK AT25 QSPI_nCS AT24	LVC MOS33	SST26VF064B, owned by the DUT
User microSD	USD_CLK AU15 USD_CMD AU16 USD_DAT[3:0] AV14 AV13 AT13 AT12 USD_NCD AT15	LVC MOS33	SPI mode: the OS slots, <code>/persist</code> and the design store (§9.3)

GROUP	PORTS AND PACKAGE PINS	I/O STANDARD	PURPOSE
DDR4	117 pins: <code>c0_ddr4_*</code> (17 address, 2 bank, 1 bank-group, command and control, 64 data bits with 8 <code>dm_n</code> and 8 differential <code>dqs</code> pairs) and <code>c0_sys_clk_p</code> H19 / <code>c0_sys_clk_n</code> H18. The full list is <code>fpga/shell/constraints/mbv/ddr4_pins.xdc</code>	Set by the DDR4 controller IP	The MicroBlaze V's main memory (§7.6)
MCC-facing	<code>CFG_DATAOUT</code> AV18 <code>IOFPGA_SYSWDT</code> AU20 <code>SMBM_nWAIT</code> AP28 <code>WDOG_RREQ</code> AU19	LVC MOS33	Tied off in the shell

A design's own pins (its QSPI flash and GPIO) are reached through the boundary (§10.1) and constrained by the shell, not by the design. The DDR4 pins are constrained only when the shell is built with `SHELL_CPU=mbv`. The four MCC-facing pins are tied off: the harness has no data path to the MCC (§7.6).

B The address map

The MicroBlaze V's view of the static shell, as generated from the block design by `tools/gen_regmap.py`. Each block has exactly one owner under Linux: the kernel, through a driver, or `harnessed`, through a userspace I/O mapping. The contract `docs/contracts/shell-regmap.md` has every register (19 blocks, 110 registers) and its bit fields, and `fpga/shell/generated/regmap_mbv.json` the same map for machines.

BASE	SIZE	BLOCK	OWNER	PURPOSE
<code>0x0000_0000</code>	128 KiB	LMB	stage-0, <code>harnessed</code>	Local memory: stage-0 (reset vector), its status block at <code>0x1_FE00</code> , the diagnostic mailbox at <code>0x1_FF00</code>
<code>0x4060_0000</code>	64 KiB	UARTLITE	kernel	The harness serial console, <code>ttyUL0</code> (interrupt 2)
<code>0x4120_0000</code>	64 KiB	INTC	kernel	The interrupt controller
<code>0x41C0_0000</code>	64 KiB	TIMER	kernel	The kernel's clock events (interrupt 1)
<code>0x44A0_0000</code>	64 KiB	CLKRST	<code>harnessed</code>	Design, partition and debug resets; DUT clock status
<code>0x44A1_0000</code>	64 KiB	DFXCTL	<code>harnessed</code>	The decoupler; the live <code>rm_id</code> and its status
<code>0x44A2_0000</code>	64 KiB	HWICAP	<code>harnessed</code>	AMD AXI HWICAP (PG134), FIFO mode: partial and clearing bitstreams (interrupt 0)
<code>0x44A3_0000</code>	4 KiB	VPHY	<code>harnessed</code>	The design's virtual Ethernet PHY: link state and MDIO model
<code>0x44A4_0000</code>	64 KiB	USD	kernel	The user microSD in SPI mode, through <code>mmc_spi</code> (<code>mmcblk0</code>)
<code>0x44A5_0000</code>	64 KiB	TELEM	<code>harnessed</code>	<code>STATUS[0]</code> the DDR4 calibrated; no power sensor
<code>0x44A6_0000</code>	4 KiB	GENCHK	<code>harnessed</code>	Traffic generator and checker for the design's Ethernet; its sniffed IP address
<code>0x44A7_0000</code>	64 KiB	JTAGBB	<code>harnessed</code>	JTAG bit-bang to the design, behind TCP 6921
<code>0x44A8_0000</code>	64 KiB	DBGBR	<code>harnessed</code>	AMD Debug Bridge (PG245), behind XVC 2542
<code>0x44A9_0000</code>	64 KiB	UARTBR	<code>harnessed</code>	The design's console and SWO byte FIFOs, behind 6930–6932
<code>0x44AA_0000</code>	64 KiB	GPIO	<code>harnessed</code>	Board LEDs and switches, with per-bit harness override; heartbeat
<code>0x44AB_0000</code>	64 KiB	MMCM_DRP	<code>harnessed</code>	The DUT clock's MMCM, retuned at run time (PG065)
<code>0x44AC_0000</code>	64 KiB	CLCD	<code>harnessed</code>	The panel's 8080 bus
<code>0x44AD_0000</code>	64 KiB	CLCDKVM	<code>harnessed</code>	The panel's hand-over between harness and design; the display tunnel; backlight
<code>0x44AE_0000</code>	64 KiB	TOUCH	<code>harnessed</code>	AMD AXI IIC (PG090) to the STMPE811 touch controller, polled
<code>0x44B2_0000</code>	64 KiB	DUTEGR	<code>harnessed</code>	Capture of the frames the design transmits (and an inject side)
<code>0x44B3_0000</code>	64 KiB	USRACC	<code>harnessed</code>	The bitstream's <code>USR_ACCESS</code> stamp
<code>0x44B4_0000</code>	64 KiB	WDOG	<code>harnessed</code>	AMD AXI Timebase Watchdog (PG128), kicked by the service loop
<code>0x8000_0000</code>	1 GiB	DDR4	kernel	Main memory, through the MIG in SLR1; cached
<code>0xC000_0000</code>	16 MiB	LAN9220	kernel	<code>eth0</code> , through <code>axi_emc</code> (interrupt 3)

`0x44AF_0000` – `0x44B1_0000` are reserved. The `TOUCH` block exists only in a shell built with `SHELL_TOUCH=1`, as the release's is. The MicroBlaze V runs `rv32imac_zba_zbb_zbs` with Sv32 at 100 MHz, with its reset vector at `0x0000_0000`.

C The bare-metal shell (v1.x)

Releases v1.0.0 and v1.1.0 used a bare-metal shell: static `0x72BB0A36` (v1.1.0), a classic MicroBlaze running C firmware v0.11, built with Vivado 2024.1. It is built from the same block design with `SHELL_CPU=mb`, so the partition, the boundary and the blocks at `0x44A0 – 0x44B4` are the same. It is frozen; it matters now only as the shell a board may be rolled back to, or found running.

	LINUX HARNESS (V2.0)	BARE-METAL SHELL (V1.X)
Processor	MicroBlaze V with MMU, 1 GiB DDR4	MicroBlaze v11, no MMU, no DDR4; 1 MiB local memory holds the whole firmware
Software	Stage-0, Linux, <code>harnessd</code> , SSH	One cooperative C loop on lwIP
Start-up	3 to 4 minutes	Seconds, as soon as configuration ends
Updating it	A Linux image into an OS slot	Firmware baked into the bitstream with <code>updatemem</code> , then a config-SD write
Loads	About a minute; any static-matching overlay	7 to 18 s; only overlays keyed to <code>0x72BB0A36</code>
User microSD	OS slots, <code>/persist</code> , the design store	Not used: always powers up on <code>greybox</code>
Identity	Per board (chapter 3)	One for every board: <code>MPS3-01</code> , <code>192.168.10.101</code> , <code>02:00:00:4D:50:53</code>
Debug	OpenOCD on the board, through SSH	An OpenOCD on the PC with <code>remote_bitbang</code> , straight to 6921

Protocol v0.11 is a subset of v0.17 on the same ports: no identify (6899), no SSH, no panel mirror and no claim, so JTAG and XVC are open to the network; the 6910 push must use the windowed variant, which tools choose from `version.features`; `usd`, `slot`, `identity` and `locate` are refused; `version` has no `impl` key. If `identify` times out but `ping` answers, the board runs the bare-metal shell. Harness Manager and `pyverify` work with both.

Rolling a board back to v1.x is a config-SD install of the v1.x release's bundle and an MCC reboot (§1.3). It takes under 15 minutes and leaves the user microSD untouched, so installing the v2.0 bundle again brings the Linux harness back. The v1.x shell loads only the v1.x overlay set; tell users first.

Building it needs Vivado and Vitis 2024.1. Its firmware is built from the build's `.xsa` with `make -C firmware/platform elf PRODUCT=1`: `PRODUCT=1` is the flag set the release used, and a bare `make elf` gives an image that pings, passes every check, and cannot load a design. A firmware-only change is baked into the existing base with `updatemem`, keeping the `static_id`.

D Platform-developer troubleshooting

Symptoms met by people who build, change or script the harness, rather than use it. Each has cost someone a session, and most looked like a hardware fault or somebody else's bug. Users' problems are in *User Guide* ch 9; the recovery ladder is chapter 5.

D.1 The harness

SYMPTOM	CAUSE	FIX
The board reboots about 43 s after a JTAG script ran	The script halted the MicroBlaze V: Linux stopped, and the watchdog reset the board	Never halt the harness's processor; read its state over SSH (<code>pyverify mailbox</code>)
The first load after a restart finds the design dead	After any harness restart the partition stays in reset until the first load	Load <code>greybox</code> once after every restart before judging anything
Loads refused with <code>identity lock: ...</code> , or <code>identify</code> says <code>nohw</code>	The Linux image and the base bitstream were built for different statics	Push the image built for this base, or install the base the image was built for
A connection to 6900, 6910, 6921 or 2542 is reset at once	Each port takes one client, and the previous connection, often your own, has not been reaped yet	Treat it as busy, not down: retry after a few seconds
<code>mps3-debug up</code> exits 4 or 6	4: another client holds 6921, named in <code>busy</code> . 6: OpenOCD exited	4: stop that client (a PC OpenOCD or a forward left open). 6: read <code>/run/mps3-debug/openocd.log</code>
A tool misjudges the board from <code>identify</code> 's <code>proto</code>	<code>proto</code> lags the protocol version	Test <code>version</code> 's features by name
A tool says "ok" and the board has not changed	The tool reported what it sent, not what the board did	Confirm every change from the board: <code>ping</code> , <code>identify</code> , <code>stats</code>
The harness serial console is silent	The wrong port (it is the third of the four), or another program has it open	Open the third port at 115 200 baud, alone; or read <code>pyverify log</code>

D.2 Scripts and sessions

SYMPTOM	CAUSE	FIX
A PC OpenOCD cannot use the board's JTAG port	Its build has no <code>remote_bitbang</code> adapter	Use a build that has it (xPack OpenOCD 0.12), or the board's own OpenOCD
After a failed PC OpenOCD run, the next session is refused	The SSH forward still holds a client on 6921	Close the forward too, then start again
A <code>pskill -f</code> kills the script that ran it, or a remote <code>pgrep -f</code> always matches	The pattern appears in the command line of the shell running it	Anchor the pattern to the real command (<code>^/usr/bin/python3 ...</code>), or keep a pid file
A script opens another board's JTAG cable	A shared <code>hw_server</code> lists every cable, and "the first target" belongs to another board	Filter by the cable's serial and refuse unless exactly one target matches (<code>scripts/mps3_hw_target.tcl</code>)
A config-SD write "timed out", and a retry left the board dark	The write outlasts the client; the second write corrupted the first	Write once, wait, confirm (§6.5). Restore the backup (§5.2, rung 7)

D.3 The nanoSoC designs and the design's flash

SYMPTOM	CAUSE	FIX
<code>nanosoc</code> locks up (<code>pc 0xfffffffffe</code>) on a board whose flash holds MicroPython; with a blank flash it runs its hello program	Known issue 14. <code>rm_nanosoc</code> bakes the flash-boot stage-0 ROM: it finds MicroPython's boot table, copies a hot section of about 40 KB into its 16 KB of instruction memory, which aliases, fails the CRC and jumps to a clobbered vector	Load <code>nanosoc_upy</code> on a board whose flash holds MicroPython. In the repository, build <code>rm_nanosoc</code> with <code>QSPI_FLASH_PRESENT=0</code>
MicroPython stops starting after <code>nanosoc_multicore</code> ran; bytes from <code>0x20000</code> read <code>0x00</code>	Known issue 15. Core 1's boot ROM writes one byte at flash <code>0x20000</code> on every boot, the start of MicroPython's cold section. Harness Manager asks for <code>MULTICORE</code> to be typed before it loads it	Write the MicroPython image again; keep <code>nanosoc_multicore</code> off boards whose flash holds it
OpenOCD reports <code>Cortex-M PARTNO 0x0</code> is unrecognized for <code>nanosoc_multicore</code> 's core 0	Core 0 stays in reset until core 1's boot ROM opens its boot gate, after a valid flash table	Debug core 1; open the gate through access port 1 before examining core 0 (not proven on the board)